Relazione progetto
Cognitive Computing and Artificial Intelligence

Messina Gioele Luca Camiolo

2025/2026

Indice

1 Introduzione **3**

2 Struttura dell'Agente **3**
2.1 TRIAGE_UNIT (Sottografo) 4
2.2 ROUTER_AFTER_TRIAGE 6
2.3 LLM_CALL 7
2.4 MERGE_RESULT 8
2.5 FACT_CHECK_NODE 8
2.6 HITL_POST & REFINE_PROPOSAL 9
2.7 CREATE_POST_NODE_IN_NEO4J 10
2.8 SUGGESTION_SUBGRAPH (Sottografo) e Pianificazione Suggerimento . 10
2.9 PLANNING_SUBGRAPH (Sottografo) e Pianificazione Editoriale 11

3 Tool **14**
3.1 `Tavily_search_tool.py` 15
3.2 `Steam_tool.py` 15
3.3 `RAG_tool.py` 17
3.4 `Neo4j_tool.py` 17

4 Prompt ed Altri File **18**

5 MCP (Model Context Protocol) **19**

6 Knowledge Graph (Neo4j) **19**

7 RAG (Retrieval-Augmented Generation) **25**

8 Esecuzione Agente **27**
8.1 Flusso Esecutivo per la Scrittura di una Recensione/News 28
8.2 Flusso Esecutivo per il Suggerimento Editoriale 33
8.3 Flusso Esecutivo per la Pianificazione Editoriale 36

9 Fine Tuning **39**

9.1 Definizione del Problema e Motivazione . . . 39
9.1.1 Design Sperimentale . . . 40
9.1.2 M1 — LoRA su Wenfi (Transfer da film a videogiochi) . . . 40
9.1.3 M2 — Full Fine-Tuning DistilBERT . . . 40
9.1.4 M3 — Zero-Shot RoBERTa-MNLI (Baseline) . . . 40
9.1.5 BERT-base — Check Architetturale . . . 41
9.2 Dataset . . . 41
9.2.1 Sorgente . . . 41
9.2.2 Pulizia e Bilanciamento . . . 41
9.3 Criteri di Valutazione e Selezione degli Iperparametri . . . 42
9.3.1 Metrica Principale . . . 42
9.3.2 Ricerca Iper-parametri . . . 42
9.4 M3 — Zero-Shot Baseline (RoBERTa-large-MNLI) . . . 43
9.5 M1 — LoRA su DistilBERT (Checkpoint Wenfi/IMDB) . . . 44
9.5.1 Valutazione Wenfi Prima dell'Adattamento . . . 44
9.5.2 Fine Tuning LoRA . . . 44
9.5.3 Risultati Finali M1 sul Test Set . . . 45
9.6 M2 — DistilBERT Full Fine-Tuning . . . 46
9.6.1 Valutazione DistilBERT prima del Tuning . . . 46
9.6.2 Valutazione DistilBERT Full Fine-Tuning . . . 46
9.6.3 Risultati Finali M2 sul Test Set . . . 47
9.7 BERT-base-uncased — Confronto Architetturale . . . 48
9.7.1 Valutazione BERT prima del Tuning . . . 48
9.7.2 Valutazione BERT Full Fine-Tuning . . . 48
9.7.3 Risultati Finali BERT-base sul Test Set . . . 49
9.8 Efficienza Parametrica: LoRA vs Full Fine-Tuning . . . 50
9.9 Confronto Complessivo . . . 51
9.10 Stress Test su Esempi Scritti a Mano . . . 52
9.11 XAI — Explainable AI con Integrated Gradients . . . 53
9.11.1 Implementazione . . . 53

10 Analisi dei Risultati e Conclusioni Finali **55**

1 Introduzione

Il progetto sviluppato consiste in un assistente editoriale agentico specializzato in ambito videoludico, progettato per supportare l'autore del blog durante l'intero ciclo di vita della produzione e pubblicazione dei contenuti. Il sistema è in grado di proporre nuove tematiche per news e recensioni analizzando lo storico degli articoli già pubblicati, generare piani editoriali strategici motivandone le scelte e assistere nella stesura delle bozze attraverso l'integrazione di ricerche sia sul web sia su fonti locali attraverso l'uso del RAG. A supporto dell'intero flusso operativo è stato realizzato un Knowledge Graph, che consente di archiviare, organizzare e mettere in relazione i contenuti approvati, favorendo una gestione strutturata della conoscenza e il riutilizzo delle informazioni nel tempo.

Per preservare il controllo editoriale da parte dell'autore, il sistema adotta un paradigma **Human-in-the-Loop**, in cui ogni operazione significativa richiede una validazione umana prima di essere eseguita. Questo approccio garantisce che tutte le decisioni rilevanti, inclusa la pubblicazione dei contenuti, siano sempre supervisionate e approvate dall'autore del blog.

Scelte progettuali

Per quanto riguarda la piattaforma di sviluppo, è stato utilizzato Visual Studio Code in combinazione con il framework di orchestrazione a grafo **LangGraph**, sviluppato ad hoc per la progettazione, l'esecuzione e l'osservazione di architetture agentiche.

L'architettura della memoria del sistema si basa sull'integrazione di due tecnologie distinte:

- **Neo4J**: un servizio eseguito in locale per l'implementazione del Knowledge Graph (database a grafo), all'interno del quale vengono memorizzati e strutturati tutti i post pubblicati.
- **ChromaDB**: un database vettoriale open-source utilizzato nella componente RAG (Retrieval-Augmented Generation) per il salvataggio dei documenti locali sotto forma di embeddings vettoriali.

All'interno del sistema agentico è stato inoltre implementato il **Model Context Protocol (MCP)**, uno standard open-source introdotto da Anthropic che permette agli LLM di connettersi in modo sicuro e standardizzato a servizi e strumenti (tool) esterni.

Infine, l'attività di fine-tuning dei modelli è stata interamente esternalizzata su **Kaggle**. Questa piattaforma ha fornito l'ambiente di sviluppo e le risorse computazionali cloud (GPU) necessarie per l'addestramento, superando così l'assenza di hardware dedicato in locale.

2 Struttura dell'Agente

L'architettura dell'agente è stata strutturata in **sottografi**, una scelta progettuale legata alla modalità con cui i nodi comunicano all'interno di LangGraph. In questo framework, il passaggio di informazioni avviene attraverso lo *State* (lo stato dell'esecuzione), il quale aggiorna il campo `messages` a ogni singola azione o risultato ottenuto dal large language model.

Organizzare l'intero sistema sotto un unico grafo avrebbe comportato un problema critico: l'accumulo incontrollato di cronologia nel campo `messages`. Questo fenomeno non solo porta alla rapida saturazione della context window dell'LLM, ma genera anche interferenze cognitive per il modello. Poiché l'agente adotta il paradigma **ReAct** (Reason and Act), esso ha la necessità di elaborare i risultati precedenti per prendere decisioni successive; la presenza di messaggi ridondanti, legati ad altri task o generati da altri LLM, finirebbe per confondere il modello, compromettendone l'accuratezza. Per ovviare a questo limite, si è optato per la scomposizione in sottografi. Questa architettura consente di isolare e differenziare gli stati per ogni specifico compito: ogni sottografo mantiene uno stato indipendente e, nel passaggio da un modulo all'altro, vengono trasferiti esclusivamente i messaggi strettamente necessari all'avanzamento del flusso editoriale. Nella Figura 1, è riportata la struttura del sistema agentico, la cui implementazione è definita nel file `workflow.py` presente nella directory principale del progetto.

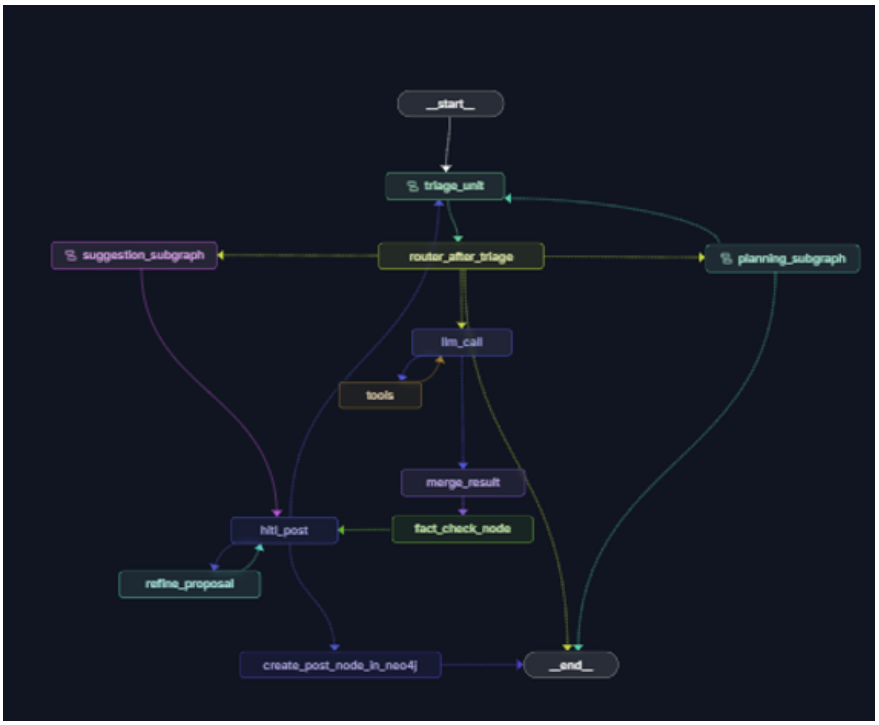


Figura 1: Struttura a grafo del sistema agentico.

Come si vede nella Figura 1, il flusso esecutivo del sistema può diramarsi in tre direzioni differenti in base alla tipologia di richiesta effettuata dall'utente (richiesta di suggerimento, richiesta di pianificazione oppure richiesta di realizzazione di una determinata recensione/news). Indipendentemente dal percorso intrapreso, l'input iniziale viene sempre intercettato ed elaborato dal sottografo `triage_unit`.

2.1 TRIAGE_UNIT (Sottografo)

Questo sottografo è composto da tre nodi principali che condividono il medesimo stato, denominato `TriageState`. A livello di architettura del codice, la logica è organizzata all'interno della cartella `Triage_router`, suddivisa nei seguenti file:

- `triage_router.py`: contiene la logica del nodo di routing iniziale

- `clarification_user.py`: contiene la logica del nodo `clarify_and_write_brief`
- `workflow_triageRouter.py`: definisce la struttura, i collegamenti e i passaggi del sottografo

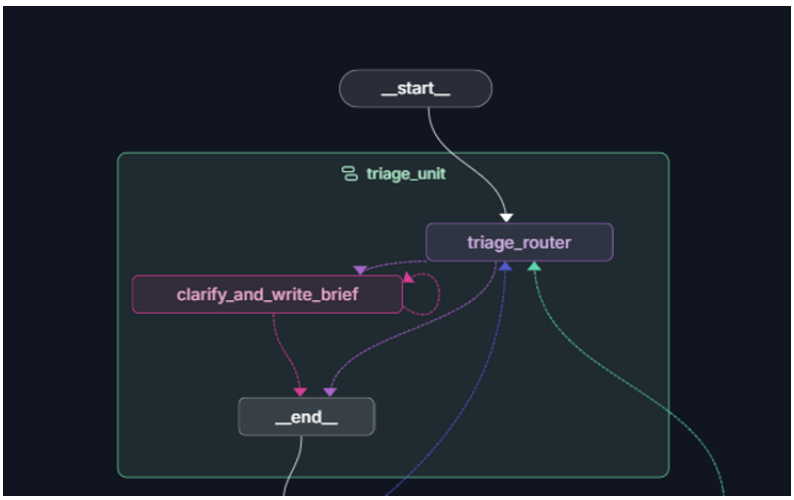


Figura 2: Struttura del sottografo `triage_unit`.

Quando la richiesta iniziale entra nel nodo `triage_router`, un LLM la classifica in una delle seguenti categorie: **News**, **Review**, **Suggestion**, **Planning** o **Ignore**.

A seconda della classificazione, il sistema si comporterà in modo differenziato:

1. **Ignore**: Qualsiasi richiesta non pertinente all'ambito videoludico viene etichettata sotto questa categoria e l'esecuzione viene immediatamente interrotta.
2. **Planning o Suggestion**: Il flusso bypassa il nodo `clarify_and_write_brief` e punta direttamente alla fine del sottografo, confluendo nel nodo successivo `router_after_triage`.
3. **News o Review**: La richiesta viene instradata verso il nodo `clarify_and_write_brief`. Il compito di questo nodo è verificare che l'input dell'utente non sia troppo generico.
    - *Se la richiesta necessita di dettagli:* si attiva un meccanismo **HITL (Human-in-the-Loop)** per porre domande di chiarimento all'utente. Una volta raccolte le specifiche mancanti, l'LLM riformula la richiesta iniziale integrandovi i nuovi dettagli strutturati.
    - *Se la richiesta è già ben formata:* il sistema non richiede alcun intervento.

Raggiunta l'esaustività necessaria della richiesta, il nodo interroga immediatamente il Knowledge Graph Neo4j tramite chiamata diretta di tool specifici a seconda del tipo di richiesta (`check_game_already_reviewed_tool` o `check_news_already_published_tool`).

Qualora il database intercetti dei contenuti preesistenti sul medesimo argomento, il sistema non avvia un'interruzione immediata, ma introduce una fase intermedia di validazione semantica affidata a un LLM con funzioni di "Giudice Editoriale". A questo modello viene sottoposto un quadro informativo completo che include: il nome ufficiale del gioco (`game_name`), la richiesta dell'utente, eventualmente arricchita con i chiarimenti richiesti (`research_brief`), e i record grezzi restituiti dal database.

Il blocco Human-in-the-Loop (HITL) viene innescato esclusivamente se l'LLM conferma che si tratta di un "vero duplicato", scartando autonomamente i falsi positivi, ovvero quei

contenuti che trattano lo stesso gioco ma con focus o formati editoriali differenti. In caso di duplicato confermato dall'IA, l'interfaccia mostra all'utente non solo i match del database, ma anche il ragionamento logico espresso dal modello a supporto della sovrapposizione rilevata. L'utente può così decidere se annullare definitivamente l'operazione (terminando il grafo e impostando la classificazione su *"Ignore"*) oppure forzare la generazione del contenuto. Di seguito è riportato il rilevamento di un post già pubblicato precedentemente che rispecchia la richiesta sottoposta dall'utente.

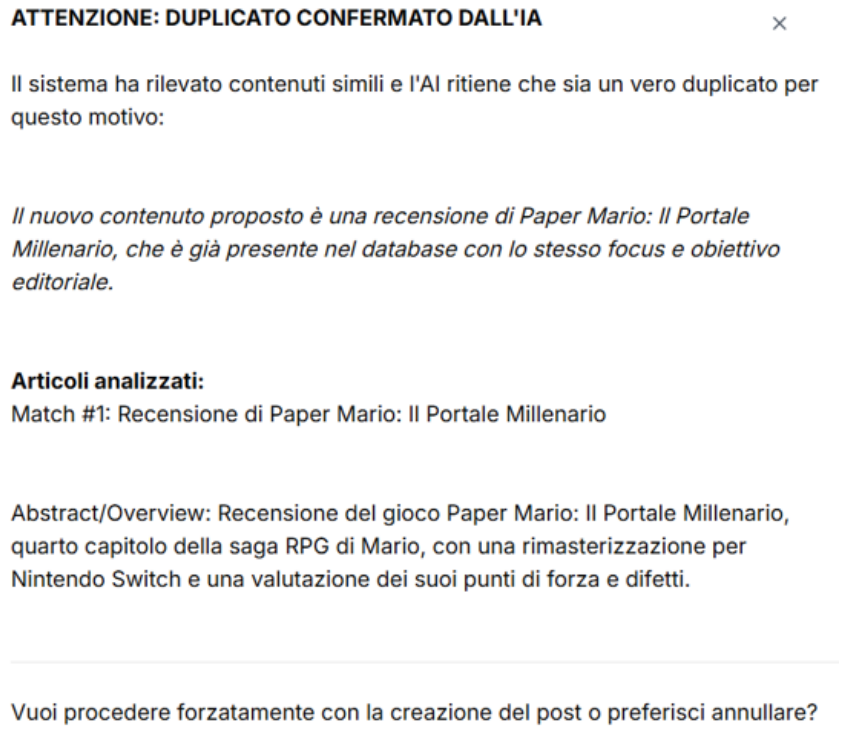


Figura 3: Interfaccia HITL per la notifica di duplicato confermato dall'IA.

2.2 ROUTER_AFTER_TRIAGE

Questo nodo, la cui implementazione è definita nel file `route_after_triage.py`, agisce come un componente di smistamento (routing) strategico. Basandosi sulla classificazione precedentemente effettuata dal sottografo `triage_router`, indirizza il flusso esecutivo verso tre direzioni distinte:

- **Suggestion**: La richiesta viene reindirizzata ed elaborata verticalmente all'interno del sottografo dedicato `suggestion_subgraph`.
- **Planning**: Il flusso viene trasferito al sottografo `planning_subgraph` per l'elaborazione dei piani editoriali e delle sequenze temporali dei post.
- **News o Review**: Qualora la richiesta dell'utente esprima la volontà diretta di redigere una news o una recensione specifica, il flusso non devia su sottografi verticali ma prosegue sul grafo principale. In questo scenario, l'input viene inoltrato direttamente al nodo centrale di reasoning dell'intera architettura: il nodo `llm_call`, che funge da core decisionale del sistema.

2.3 LLM_CALL

Questo componente rappresenta il "cervello" decisionale del grafo principale e ha il compito di analizzare lo stato corrente dell'esecuzione per determinare autonomamente la strategia migliore da adottare. La sua logica operativa, definita all'interno del file `reasoner.py`, si sviluppa attraverso tre fasi principali:

- **Valutazione del Contesto e Dynamic Prompting**: All'ingresso del nodo, il sistema recupera la cronologia dei messaggi e la richiesta dell'utente. In base alla classificazione precedentemente effettuata dal router (News o Review), viene iniettato dinamicamente un system prompt specializzato (rispettivamente `research_agent_prompt_News` o `research_agent_prompt_review`), arricchito con metadati temporali in tempo reale (la data odierna). Questo garantisce che il modello ragioni sempre con la massima consapevolezza del contesto editoriale e temporale.
- **Strategia Agentica e Selezione dei Tool**: Il modello linguistico (LLM) analizza le informazioni a sua disposizione e decide, secondo il paradigma ReAct, se la base di conoscenza attuale è sufficiente per completare il task o se è necessario raccogliere ulteriori dati. Per farlo, ha accesso a un set di strumenti (tool) specializzati sia locali sia esterni (le implementazioni dei tool sono organizzate in file all'interno della cartella `tools`), che includono:
  - `Tavily_search_tool`: Tool che permette di fare ricerche su internet. Tramite questo tool l'LLM ricerca informazioni su internet riguardo l'argomento richiesto per poter scrivere la bozza finale della news/recensione. Il Tool è messo a disposizione tramite il server MCP.
  - `RAG_tool`: Interrogazioni della base documentale affidabile locale (RAG) per recuperare ulteriori informazioni ritenute completamente affidabili.
  - `Recensioni_utenti_steam` e `ottieni_prezzo_gioco`: due tool messi a disposizione dalla piattaforma online Steam per poter recuperare prezzo, recensioni positive e negative del gioco. Questi tool vengono utilizzati ovviamente solo nel caso in cui la richiesta riguarda la recensione di un videogioco ed anche in questo caso vengono utilizzati solamente se LLM capisce che il gioco è uscito anche per PC, altrimenti non vengono usati in quanto non restituirebbero nessuna informazione.
  - `Cerca_post_gioco`: tool messo a disposizione per ricercare all'interno del Knowledge Graph Neo4J post già pubblicati relativi all'argomento richiesto o pertinente all'argomento, in modo da poter andare a fare collegamenti con post precedenti e consigliarne la lettura.
- **Routing Post-Esecuzione (`route_after_llm`)**: Ogni volta che il nodo `llm_call` emette un output, un meccanismo di controllo condizionale (`route_after_llm`) esamina la risposta:
  - Se l'LLM ha ritenuto necessario invocare uno o più strumenti, il flusso viene instradato verso il nodo di esecuzione dei tool (`ToolNode`), disabilitando le chiamate parallele per mantenere un tracciamento deterministico e sequenziale delle azioni.
  - Se il modello ritiene di possedere tutti gli elementi necessari per elaborare il testo finale (news o recensione), salta la fase di ricerca e indirizza il flusso verso

il nodo di consolidamento del risultato `merge_result`.

2.4 MERGE_RESULT

Questo nodo, la cui implementazione è definita nel file `merge_result.py`, agisce come nodo di unificazione e sintesi finale del grafo principale. Il suo obiettivo primario è raccogliere i dati restituiti dai vari tool invocati dal nodo precedente (`llm_call`) durante la fase di raccolta delle informazioni, strutturarle e invocare il modello linguistico per la generazione della bozza editoriale conclusiva. La sua logica operativa si sviluppa attraverso tre fasi sequenziali:

1. **Estrazione e Categorizzazione dei Risultati dei Tool:**
   Il sistema analizza l'intera cronologia dei messaggi (`messages`) memorizzata nello stato dell'agente per isolare esclusivamente i messaggi di tipo `ToolMessage` attraverso il metadato `name` di ciascun messaggio per identificare quale strumento lo ha generato.
2. **Generazione Dinamica del Riassunto (Summarization):**
   Una volta elaborate tutte le informazioni raccolte viene invocata la funzione di supporto `summarize_webpage_content`. Questa funzione, sulla base del tipo di richiesta (news o recensione) applica una personalizzazione del comportamento dell'LLM, in particolare effettua una selezione del Prompt da fornire all'LLM in base al Topic. Il sistema controlla la classificazione originaria salvata nello stato (`classification_decision`), se il topic è identificato come "News", viene iniettato il template di prompt specializzato `summarize_webpage_news`, in caso di recensione viene adottato il template `summarize_webpage_gameReview`.
3. **Aggiornamento dello Stato:**
   Al termine del processo, il nodo restituisce un dizionario strutturato che aggiorna lo stato globale dell'agente (`AgentState`):
   - `merge_result`: Contiene il testo finale della news o della recensione, completamente strutturato, riassunto e formattato dall'LLM.
   - `link_source`: Fornisce una lista pulita di tutte le fonti web esterne consultate. La lista viene preventivamente elaborata tramite una conversione a set (`list(set(links))`) per garantire l'eliminazione nativa di qualsiasi URL duplicato, offrendo all'utente finale un elenco trasparente e verificabile di riferimenti.

Una volta terminata l'esecuzione di questo nodo il flusso procede verso il nodo `fact_check_node`.

2.5 FACT_CHECK_NODE

Questo componente, definito nel file `fact_checker.py`, agisce come nodo di controllo qualità e allineamento informativo del grafo principale. Il suo scopo fondamentale è analizzare la bozza generata (`merge_result`) e confrontarla direttamente con i dati grezzi estratti dai messaggi dei tool di ricerca (`tavily_search_tool` e `RAG_tool`), al fine di intercettare e segnalare la presenza di eventuali "allucinazioni" o discrepanze fattuali nella bozza proposta.

Il controllo viene eseguito da un modello linguistico configurato con regole tassative iniettate tramite prompt e vincolato a un output strutturato (`FactCheckResult`). Il modello

valuta esclusivamente l'aderenza delle informazioni presenti all'interno della bozza con i dati recuperati durante la fase di ricerca effettuata dall'agente, ignorando completamente lo stile di scrittura. Qualora venga rilevato un conflitto, verificabile attraverso l'output strutturato (variabile `has_conflicts == "si"`), il sistema valorizza la variabile di stato `fact_check_warning` con una stringa di avviso formattata contenente il dettaglio delle discrepanze individuate (`discrepancies`). Se non viene riscontrato alcun disallineamento, la variabile viene impostata a `None`, validando implicitamente il testo per il nodo successivo.

2.6 HITL_POST & REFINE_PROPOSAL

Questi due componenti interdipendenti, definiti all'interno del file `hitl_post.py`, implementano il sistema Human-in-the-Loop (HITL) e il ciclo di revisione editoriale iterativo del grafo, gestendo l'interazione diretta con l'utente finale.

Il nodo `hitl_post` agisce come punto di interruzione formale del flusso tramite `interrupt`, mettendo in pausa l'esecuzione per sottoporre l'output generato alla validazione umana. Prima di mostrare la bozza (`merge_result`), il sistema antepone dinamicamente l'eventuale avviso di discrepanza generato dal fact-checker.

Il nodo `hitl_post` si occupa di gestire due tipologie di interruzione sulla base del tipo di richiesta (classificazione iniziale della richiesta). In particolare, se la richiesta è classificata come:

- **News o Review**: L'utente può approvare la bozza (indirizzando il flusso a `create_post_node_in_neo4j`), modificarla manualmente nell'editor (salvando le modifiche effettuate dall'utente e procedendo alla pubblicazione nel nodo successivo), annullare l'operazione terminando il grafo (conclude l'esecuzione), oppure tramite l'inserimento di un feedback testuale (richiedere di elaborare una porzione di testo, eliminare determinati termini ecc.) per richiedere modifiche che verranno apportate dall'AI. In quest'ultimo caso, il flusso devia verso il nodo di correzione `refine_proposal`.
- **Suggestion**: All'utente in questa situazione non viene sottoposta una bozza editoriale ma gli viene sottoposto un suggerimento editoriale, che può accettare, rifiutare o modificare. Se il suggerimento viene rifiutato l'esecuzione termina, se invece viene accettato o modificato manualmente, il suggerimento diventa a tutti gli effetti una nuova richiesta utente: il grafo viene resettato e il flusso viene reinstradato al nodo iniziale `triage_unit` per avviare una nuova pipeline di ricerca e stesura.

Il nodo `refine_proposal` viene attivato esclusivamente quando l'utente richiede modifiche discorsive a una News o Review direttamente all'agente. Acquisito il feedback scritto dall'utente (salvato nella variabile di stato `user_feedback`), il nodo invoca il modello linguistico per rielaborare la bozza originale sulla base del feedback richiesto. Una volta generata la nuova versione, il nodo aggiorna lo stato globale e reindirizza automaticamente il flusso di ritorno a `hitl_post` per sottoporre la nuova bozza all'utente. Questo meccanismo instaura un loop di raffinamento continuo che persiste finché l'utente non approva la stesura definitiva.

2.7 CREATE_POST_NODE_IN_NEO4J

Questo nodo, definito in `add_node.py` insieme anche alle due funzioni di supporto utilizzate dal nodo, agisce come nodo conclusivo e di persistenza semantica del grafo principale. Il suo scopo fondamentale è estrarre dalla bozza confermata dall'utente i metadati relazionali e mappare l'intero set di informazioni all'interno del Knowledge Graph Neo4j in modo da memorizzare in maniera persistente il post pubblicato.

A seconda della classificazione della richiesta (Review o News), il nodo invoca il modello linguistico vincolandolo a uno schema di output strutturato specifico (`InfoPostReview` o `InfoPostNews`). Il compito del modello è quello di estrarre dalla proposta accettata le entità chiave, come: il titolo, i videogiochi principali e secondari menzionati (rilevandone lo stato di uscita), le case editrici, i generi di appartenenza e una sintesi descrittiva (`overview`). Il nodo processa anche l'elenco dei link grezzi (`link_source`) utilizzati per recuperare le fonti su internet isolandone i domini (`url`) per mappare correttamente le fonti e salvarle nel grafo attraverso una relazione con il post in modo da avere sempre le sorgenti da cui sono state recuperate le informazioni di ogni singolo post.

Ottenuti i dati strutturati, il nodo chiama una funzione tra `create_review_post` o `create_news_post` (a seconda del tipo di post che abbiamo creato) le quali si interfacciano con il driver Neo4j eseguendo query Cypher mirate per andare ad inserire tutte le informazioni all'interno del grafo. Il sistema genera il nuovo nodo Post (assegnandogli un UUID casuale, i contenuti testuali e la data odierna) e costruisce dinamicamente le relazioni semantiche (`ABOUT`, `MENTIONS`, `PUBLISHED_BY`, `HAS_GENERE`, `SOURCED_FROM`) con i nodi (`Game`, `Publisher`, `Genere`, `Source`), aggiornandone contestualmente le proprietà. L'esecuzione si conclude restituendo l'identificativo univoco del post creato (`neo4j_post_id`), che viene salvato nello stato dell'agente sancendo il termine del flusso.

2.8 SUGGESTION_SUBGRAPH (Sottografo) e Pianificazione Suggerimento

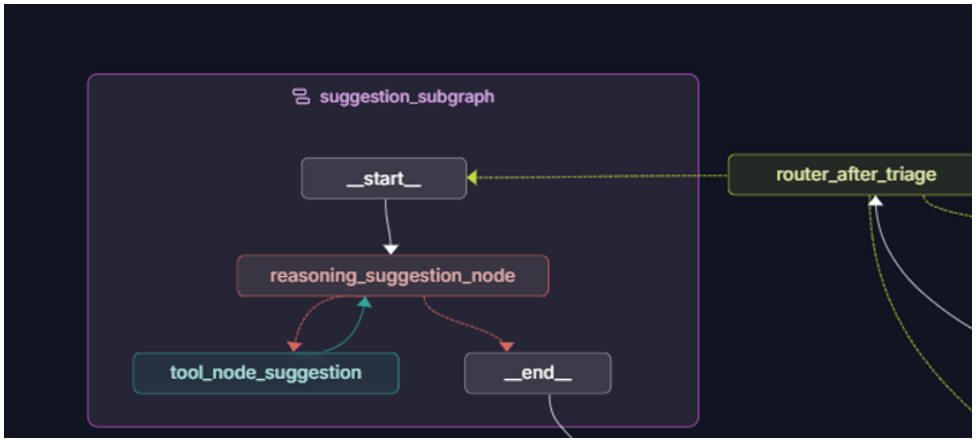


Figura 4: Struttura del sottografo `suggestion_subgraph`.

Suggestion_Subgraph

Questo sottografo, la cui architettura è illustrata in Figura 4, è dedicato alla generazione autonoma di proposte editoriali qualora l'utente richieda un consiglio su cosa pubblicare (es. "Non so cosa scrivere, potresti darmi un consiglio?").

Il sottografo è implementato all'interno della cartella dedicata `Suggestion`, all'interno della quale troviamo il file `workflow_suggestion.py` in cui è definita la topologia e la struttura del sottografo ed il file `reasoner_suggestion.py` all'interno del quale è definita la logica agentica del nodo `reasoning_suggestion_node`. L'accesso al sottografo avviene attraverso il nodo `router_after_triage` quando la richiesta dell'utente viene classificata come "Suggestion" dal `triage_router`.

Suggerimento Editoriale

Operando secondo il paradigma ReAct e coordinato dalla funzione di routing condizionale `route_after_suggestion`, il nodo principale `reasoning_suggestion_node` esegue un loop iterativo tra l'elaborazione delle informazioni in suo possesso e del pensiero prodotto, e l'invocazione sequenziale degli strumenti messi a disposizione. L'obiettivo strategico dell'LLM è formulare una proposta (News o Review) che diversifichi attivamente lo storico editoriale. Per farlo, il nodo interroga inizialmente il `get_planning_context_tool` per analizzare il bilanciamento del database (volume di news e review, ultimi generi trattati, statistiche globali) e sulla base di questo decide autonomamente la tipologia del post da proporre (recensione o news) e nel caso di scelta di una recensione, il genere del gioco da recensire. Se la scelta ricade in una:

- **Recensione**: Il modello utilizza il `get_unreviewed_released_games_tool` per cercare nel database giochi "scoperti" (menzionati in vecchi post), già rilasciati ma non ancora recensiti, che colmino le lacune di genere di gioco individuate. Se la ricerca interna non produce risultati validi, l'LLM avvia una ricerca web tramite Tavily per individuare un nuovo titolo idoneo od utilizza la propria conoscenza, validandolo obbligatoriamente con il `check_game_already_reviewed_tool`, tool che ricerca nello storico dei post pubblicati (Neo4j) se quel gioco è stato recensito in modo da evitare duplicati.
- **News**: L'agente effettua ricerche web tramite Tavily per individuare notizie attuali. Individuato l'argomento, invoca il tool `check_news_already_published_tool` per interrogare il Knowledge Graph Neo4j e accertarsi che non sia stata già coperto la medesima notizia su un post precedente.

Una volta che il modello ha individuato l'argomento definitivo da proporre, avendo superato con successo tutti i check anti-duplicazione descritti, il nodo estrae e scrive il suggerimento da sottoporre all'utente all'interno della variabile di stato `merge_result`. A questo punto, l'esecuzione del sottografo termina, ma il flusso operativo generale prosegue rendendo la proposta immediatamente disponibile per la validazione umana all'interno del nodo principale `hitl_post`.

2.9 PLANNING_SUBGRAPH (Sottografo) e Pianificazione Editoriale

Il sottografo `planning_subgraph`, la cui struttura è illustrata in Figura 5, è il sottografo dedicato alla pianificazione editoriale. L'implementazione è all'interno della cartella dedicata `Planning`, all'interno della quale troviamo il file `workflow_planning.py` in cui è definita la topologia e la struttura del sottografo, il file `reasoner_planning.py` all'interno del quale è definita la logica agentica dei nodi `planner_node`, `research_node`, `hitl_planning_node` ed il file `executer_planning.py` contenente l'implementazio-

ne dell'`executer_node`. L'accesso al sottografo avviene attraverso il nodo `router_after_triage` quando la richiesta dell'utente viene classificata come "Planning" dal `triage_router`.

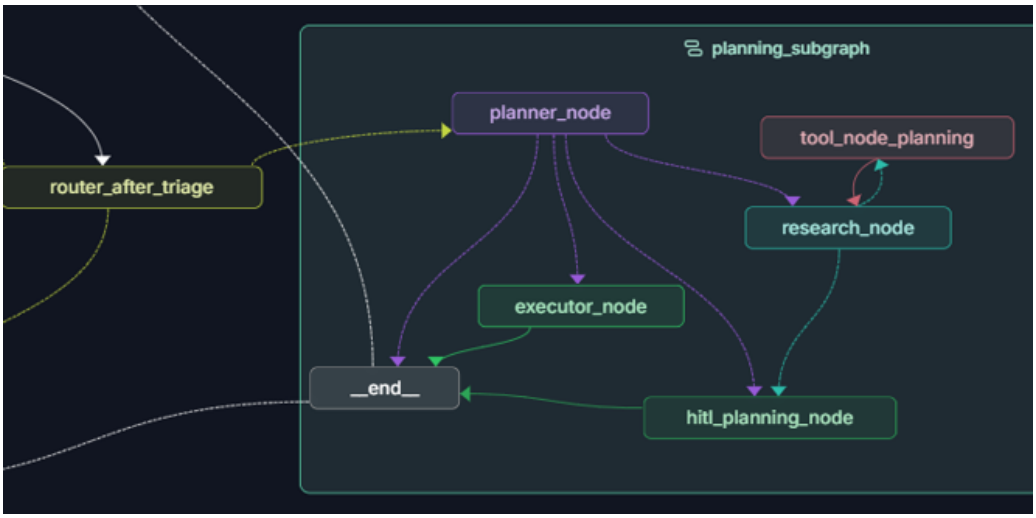


Figura 5: Struttura del sottografo `planning_subgraph`.

Pianificazione Editoriale per N Giorni

L'agente elabora un piano editoriale strutturato con una frequenza di un post al giorno. In assenza di una specifica indicazione temporale da parte dell'utente, il sistema imposta un valore di default pari a 3 giorni (3 post). Il processo di pianificazione integra una logica di bilanciamento che analizza i contenuti pregressi per evitare la ripetizione di tematiche già trattate, cercando di diversificare e bilanciare il più possibile i contenuti editoriali.

Una volta generata la proposta, l'agente sottopone all'utente il piano editoriale completo, accompagnato da una breve descrizione del ragionamento logico adottato per la selezione dei temi. Se l'utente approva la proposta, il sistema persiste la pianificazione all'interno del database a grafo Neo4j mediante l'utilizzo di nodi in stato `PENDING`, ciascuno dei quali contiene un suggerimento editoriale proposto nella pianificazione. Questa archiviazione garantisce la persistenza del piano, permettendone il recupero e la continuazione delle pubblicazioni in un qualsiasi momento. Contestualmente all'approvazione, l'agente avvia la generazione del primo post, reindirizzando il flusso al `triage_router` e quindi la pianificazione segue come descritto precedentemente per la stesura di una review/news specifica.

La progressione/cancellazione del piano editoriale è guidata dall'interazione utente:

- **Continuazione**: Per procedere con la redazione dei post successivi, è sufficiente che l'utente richieda attraverso una richiesta di proseguire la pianificazione. L'agente recupera lo stato della pianificazione in `PENDING`, identifica il post successivo da pubblicare e procede mandando in ingresso al `triage_router` la richiesta opportuna per poter procedere con la generazione della bozza per il post.
- **Eliminazione**: L'utente ha la facoltà di richiedere la cancellazione totale del piano editoriale in qualsiasi fase del processo richiedendo tramite query la cancellazione del piano editoriale. Se il piano esiste allora l'agente provvederà ad eliminarlo an-

dando ad eliminare i nodi nel database a grafo che rappresentano la pianificazione editoriale.

- **Conflitto di pianificazione**: Nel caso in cui l'utente richieda una nuova pianificazione in presenza di un piano attivo, l'agente attiva una procedura di validazione, chiedendo conferma all'utente tramite HITL per l'eliminazione dei dati preesistenti prima di sovrascriverli con la nuova proposta editoriale.

Planning_Subgraph

Il `planner_node` rappresenta il punto di ingresso del sottografo dedicato alla pianificazione editoriale. Il suo compito è interpretare la richiesta dell'utente e determinare quale azione debba essere eseguita sulla base dello stato corrente del sistema, inoltrando la richiesta al nodo opportuno che si occuperà di gestire la richiesta. In particolare, il nodo verifica se è già presente una pianificazione editoriale attiva e, sulla base della richiesta dell'utente (continuare pianificazione, eliminare pianificazione, creare pianificazione), decide l'azione che dovrà essere intrapresa (non la esegue lui) inoltrando la gestione della richiesta al nodo opportuno. Se è necessario elaborare una nuova pianificazione editoriale il flusso viene inoltrato al `research_node`; se invece è richiesta un'interazione con l'utente, ad esempio per confermare la sovrascrittura di una pianificazione esistente o per visualizzare un messaggio informativo, l'esecuzione viene trasferita al nodo `hitl_planning_node`. Se l'azione consiste nell'eseguire direttamente un'operazione sul Knowledge Graph, come l'eliminazione della pianificazione o continuare con la pubblicazione successiva della pianificazione, il controllo viene passato all'`executor_node`. Questo nodo svolge quindi esclusivamente una funzione di instradamento, separando la logica decisionale dall'effettiva esecuzione delle operazioni.

Il `research_node` è il nodo responsabile della costruzione della proposta di piano editoriale. Per farlo raccoglie tutte le informazioni necessarie interrogando il Knowledge Graph e utilizzando i tool messi a disposizione per poter recuperare tutte le informazioni di cui ha bisogno per generare la proposta. Il suo obiettivo è individuare quali e quanti contenuti tra news e recensioni proporre nel piano editoriale in modo da bilanciare il più possibile i contenuti all'interno del blog, evitando argomenti già trattati e privilegiando, nel caso delle recensioni, i videogiochi già presenti nel backlog ma non ancora recensiti. Al termine della fase di ricerca produce una proposta completa di pianificazione accompagnata dal ragionamento che ha guidato le scelte effettuate.

Il nodo `hitl_planning_node` gestisce tutte le interazioni tra il sistema e l'utente durante la pianificazione editoriale. In particolare, presenta eventuali messaggi informativi, le richieste di conferma per operazioni che richiedono l'intervento dell'utente, come la cancellazione o la sostituzione di una pianificazione già esistente, e la proposta del piano editoriale generato.

L'utente può accettare la pianificazione, modificarla manualmente oppure annullare l'operazione. Il nodo gestisce inoltre le situazioni in cui viene richiesta un'operazione non eseguibile, ad esempio il tentativo di proseguire o eliminare una pianificazione inesistente, fornendo all'utente un opportuno messaggio informativo prima di terminare il flusso.

L'`executor_node` è responsabile dell'esecuzione delle operazioni sul Knowledge Graph richieste dal sottografo di pianificazione per aggiornare lo stato della pianificazione editoriale.

Nel caso in cui venga approvata una nuova pianificazione, il nodo estrae automaticamente i singoli contenuti previsti dal piano editoriale e li memorizza nel Knowledge Graph come nodi di tipo `Pianificazione` con stato `PENDING`, preservandone l'ordine di esecuzione. Contestualmente avvia immediatamente la produzione del primo contenuto previsto, inoltrando la relativa richiesta in ingresso al `triage_unit`.

Qualora l'utente richieda di proseguire una pianificazione e questa esiste già, il nodo recupera l'elemento successivo ancora in stato `PENDING`, lo rimuove e lo inoltra al sottografo principale `triage_unit` affinché venga generata la bozza del post.

Il nodo gestisce inoltre la cancellazione di una pianificazione editoriale quando l'utente lo richiede, eliminando tutti i nodi ancora in stato `PENDING`.

Nel caso in cui l'utente richieda la creazione di una nuova pianificazione in presenza di una precedente già esistente, il nodo provvede prima ad informare l'utente tramite HITL, se l'utente conferma l'azione proseguirà con la rimozione della pianificazione corrente e successivamente riavvia il processo di ricerca per la generazione del nuovo piano editoriale.

3 Tool

Nel contesto delle architetture basate su Large Language Models, l'implementazione degli strumenti (o tools) rappresenta il meccanismo fondamentale attraverso cui il sistema supera i propri limiti intrinseci. Un modello linguistico puro è, per sua natura, un'entità statica: le sue conoscenze sono vincolate alla propria conoscenza acquisita dal dataset durante l'addestramento, quindi fortemente limitata al periodo di addestramento, risultando del tutto privo della capacità di interagire attivamente con l'ambiente circostante.

L'integrazione dei tool trasforma l'LLM da semplice generatore di testo a un vero e proprio agente decisionale e dinamico. Questi strumenti fungono da interfacce operative che il modello può invocare autonomamente per compiere azioni specifiche, recuperare dati aggiornati in tempo reale, interrogare basi di conoscenza proprietarie, validare informazioni o interagire con servizi esterni.

Operando all'interno del paradigma ReAct, l'agente analizza la richiesta dell'utente, riconosce eventuali lacune informative necessarie per l'esecuzione del task e decide strategicamente se, quando e quale strumento utilizzare. Affinché questa orchestrazione sia efficace, l'architettura richiede che ogni tool sia definito in modo rigoroso dal punto di vista programmativo:

- **Descrizione Semantica**: Ogni strumento deve possedere una definizione chiara e dettagliata. L'LLM "legge" questa descrizione per comprendere esattamente lo scopo del tool e valutarne la pertinenza rispetto al problema da risolvere.
- **Schema dei Dati**: I parametri di input richiesti dallo strumento devono essere rigidamente tipizzati. Questo permette al modello di formattare correttamente la chiamata (spesso sotto forma di JSON) e di interpretare con precisione la risposta (output) restituita dal sistema esterno.

I tool messi a disposizione dell'agente sono organizzati all'interno della cartella `tools` e suddivisi in file distinti sulla base del servizio che espongono. Complessivamente si distinguono quattro categorie, ciascuna dedicata all'interazione con uno specifico servizio: `neo4j_tool.py`, `RAG_tool.py`, `tavily_search_tool.py` e `steam_tool.py`.

3.1 `Tavily_search_tool.py`

Il file `tavily_search_tool.py` implementa il tool che consente all'agente di effettuare ricerche sul Web per reperire informazioni aggiornate non presenti nella base di conoscenza locale. A differenza degli altri tool, l'interazione con il servizio di ricerca avviene tramite un server conforme al protocollo MCP che espone il tool remoto e ne gestisce l'esecuzione.

Il tool riceve in ingresso due parametri: `query`, contenente la ricerca da fare su internet, e `ragionamento`, nel quale l'agente esplicita la motivazione che ha portato all'invocazione del tool. Una volta completata la ricerca, il contenuto restituito dal server MCP viene convertito in formato testuale e reso disponibile all'agente. Il servizio di Tavily mette a disposizione una serie di parametri configurabili in modo da gestire meglio la ricerca su internet. I parametri che sono stati configurati sono:

- `include_images=False`: disabilita il recupero di risultati contenenti immagini, limitando l'output a contenuti testuali per ridurre il rumore informativo.
- `max_results=1`: imposta il numero massimo di risultati restituiti a uno solo, privilegiando la risposta più rilevante e sintetica rispetto a un insieme più ampio di fonti. In questo modo riduciamo il numero di token utilizzati per ogni richiesta essendo che il servizio gratuito permette un numero limitato di ricerche.
- `exclude_domains=[...]`: definisce una lista di domini da escludere dai risultati della ricerca. In particolare vengono filtrati social network e piattaforme di contenuti multimediali o di e-commerce come Reddit, Instagram, YouTube, Facebook, eBay, TikTok e Dailymotion, al fine di evitare fonti non affidabili, ridondanti o non pertinenti rispetto a contenuti informativi strutturati.

Limiti Riscontrati

Sono stati riscontrati alcuni limiti relativi alla qualità delle informazioni fornite dal tool Tavily. In particolare, il contenuto recuperato presenta frequentemente una quantità significativa di rumore, dovuta alla presenza di elementi non pertinenti come testi pubblicitari, collegamenti esterni e frammenti di navigazione, riducendo la quantità e la qualità di informazioni effettivamente utili. Inoltre, il contenuto restituito consiste spesso in un riassunto molto sintetico della pagina sorgente e, in alcuni casi, risulta parzialmente troncato. Questi aspetti limitano la capacità del modello di recuperare informazioni complete e contestualizzate, aumentando il rischio che vengano considerate attendibili informazioni incomplete o non sufficientemente verificate.

3.2 `Steam_tool.py`

All'interno di questo file sono definiti una serie di tool e relative funzioni che permettono di interagire con servizi esterni messi a disposizione da Steam (piattaforma di vendita di videogiochi online per PC) attraverso delle API. Questi tool messi a disposizione al nodo `llm_call` vengono chiamati solamente nel caso in cui il videogioco è uscito su PC, altrimenti non vengono chiamati essendo che non ritornerebbero nessuna informazione dato che il gioco non sarà presente sulla piattaforma.

- `trova_id_gioco` **(funzione di utilità):**
  L'interazione con l'API di Steam richiede l'utilizzo dell'AppID, un codice numerico univoco che permette di identificare univocamente un gioco. L'utilizzo dei servizi

Steam fornendo in ingresso semplicemente il nome del gioco non è supportata ma è obbligatoriamente richiesto l'id univoco del gioco. Per questo motivo Steam mette a disposizione un servizio attraverso un endpoint che tramite il nome del gioco è possibile ottenere l'id univoco. Solamente dopo aver ottenuto l'id del gioco è possibile utilizzare i servizi per ottenere le informazioni relative al videogioco desiderato. Questa funzione è fondamentale e viene utilizzata da entrambi i tool per recuperare l'id del gioco a partire dal nome del videogioco di cui si vogliono ottenere le informazioni. Un problema che è stato rilevato nel servizio di conversione da titolo ad AppID riguarda il fatto che se si fornisce un titolo di un videogioco, il quale fa parte di una serie di videogiochi, il servizio di Steam fornisce tutti gli ID di tutti i giochi relativi a quella serie. Per questo motivo è stato necessario introdurre un LLM il quale prende in input il titolo del gioco di cui vogliamo ottenere l'id e la lista di tutte le coppie (nome gioco, AppID) ottenute dal servizio, ed in uscita, tramite l'output strutturato, fornisce l'id relativo al gioco richiesto. La scelta di utilizzare un LLM e non un classico controllo sui risultati forniti dal servizio è dovuto dal fatto che spesso nell'elenco delle coppie (nome gioco, AppID) il nome del gioco presenta caratteri speciali come (™, ©), spazi extra; quindi, un classico controllo sul nome fallirebbe.

- `ottieni_prezzo_gioco` **(tool):**
  Questo tool permette di recuperare il prezzo in tempo reale del videogioco sulla piattaforma Steam, evidenziando la presenza di eventuali sconti. Il tool richiede in ingresso il nome del gioco ed un parametro `ragionamento`, il quale serve all'LLM per indicare il motivo della chiamata al tool.

  Una volta ottenuto l'AppID, a partire dal nome del gioco invocando la funzione `trova_id_gioco`, il tool effettua una richiesta HTTP GET all'endpoint ufficiale di Steam. La configurazione della richiesta, attraverso dei parametri nell'URL dell'endpoint, assicura che le informazioni sul prezzo siano espresse nella valuta Euro e nella lingua corrette, assicurando quindi che i prezzi riflettano quelli dello store italiano in quel momento.

  Il tool restituisce in output il prezzo del videogioco in store, inoltre se presente uno sconto restituisce il prezzo scontato di acquisto ed anche la percentuale di sconto. Se il videogioco è un free-to-play (gratis) verrà restituita una variabile booleana `gratuito` a `True`.

- `estrai_recensioni` **(funzione di utilità):**
  Questa funzione di supporto è responsabile dell'interrogazione delle API di Steam per il recupero delle recensioni degli utenti. Riceve in ingresso il tipo di recensione da estrarre (positiva o negativa) e l'endpoint dell'API da interrogare. La richiesta viene configurata in modo da recuperare esclusivamente recensioni recenti, scritte in lingua italiana e limitate alle tre più rilevanti. Una volta ottenuti i risultati, la funzione esegue una fase di post-elaborazione, eliminando i caratteri di formattazione, limitando la lunghezza delle recensioni e restituendo un elenco di testi puliti. Qualora non siano presenti recensioni del tipo richiesto o si verifichi un errore durante la richiesta, viene restituito un messaggio informativo opportunamente formattato.

- `Recensioni_utenti_steam` **(tool):**
  Questo tool consente di ottenere le opinioni espresse dalla community di Steam relativamente a un determinato videogioco. Il tool come primo passaggio richiama

la funzione `trova_id_gioco` per individuare l'AppID associato al titolo richiesto. Successivamente, il tool invoca due volte la funzione `estrai_recensioni`, una per recuperare le recensioni positive e una per quelle negative. I risultati vengono infine organizzati in una struttura JSON contenente il nome ufficiale del gioco, il relativo AppID e le tre recensioni positive e negative più significative. In questo modo l'agente dispone di un insieme bilanciato di opinioni della community, utile per valutare sia i punti di forza sia le criticità del videogioco.

3.3 `RAG_tool.py`

All'interno del file è definito il tool `RAG_tool` il quale consente di interrogare la base di conoscenza locale (RAG), implementata mediante un database vettoriale Chroma, per recuperare informazioni relative ai videogiochi/news/case produttrici e molto altro. In particolare, viene utilizzato quando sono richiesti dettagli riguardanti trama, gameplay, meccaniche di gioco, analisi critica, aspetti tecnici o notizie relative a eventi e case di sviluppo.

Il tool riceve in ingresso una query in linguaggio naturale e la utilizza per eseguire una ricerca semantica all'interno del database vettoriale, sfruttando gli embedding dei documenti memorizzati. La ricerca restituisce i tre frammenti più simili alla richiesta dell'utente; tuttavia, ciascun risultato viene successivamente filtrato sulla base di una soglia minima di rilevanza, così da escludere documenti semanticamente poco pertinenti. Nella sezione relativa al RAG verranno esaminati i limiti riscontrati.

3.4 `Neo4j_tool.py`

Il file `neo4j_tool.py` raccoglie l'insieme dei tool che consentono all'agente di interagire con il database a grafo Neo4j. Tali tool vengono utilizzati per recuperare informazioni già presenti nella base di conoscenza strutturata, verificare la coerenza dei contenuti prima della loro pubblicazione e supportare le attività di pianificazione editoriale. Tutte le interrogazioni al database vengono effettuate mediante query Cypher, permettendo di sfruttare le relazioni tra i diversi nodi del grafo per ottenere informazioni contestualizzate.

- `get_planning_context_tool`: Recupera il contesto generale necessario alla pianificazione editoriale. Il tool estrae dal database statistiche relative al numero di recensioni e news pubblicate, i titoli delle ultime recensioni, i generi degli ultimi giochi recensiti, la distribuzione complessiva dei generi presenti nel database e le cinque news più recenti. Tali informazioni vengono restituite in formato JSON e costituiscono il contesto utilizzato dall'agente per pianificare nuovi contenuti.
- `check_active_plan_tool`: Verifica l'esistenza di una pianificazione editoriale ancora in corso. Il tool ricerca nel database tutti i nodi di pianificazione con stato "PENDING", li ordina secondo il campo "ordine" e restituisce l'elenco degli articoli ancora da realizzare. In assenza di una pianificazione attiva viene restituito un messaggio che informa l'agente.
- `check_game_already_reviewed_tool`: Consente di verificare se un videogioco è già stato recensito. La ricerca viene effettuata attraverso due strategie complementari: la prima verifica se il nome del gioco è presente nel campo titolo dei nodi Post di tipo Review. La seconda analizza le relazioni dei post di tipo Review e verifica se, partendo da una recensione, esista un collegamento strutturale che porti a un nodo

videogioco il cui nome corrisponda esattamente a quello richiesto. Il tool restituisce un'indicazione sull'eventuale presenza di recensioni già pubblicate, insieme alle informazioni associate, consentendo all'agente di evitare la produzione di contenuti duplicati.

- `check_news_already_published_tool`: Svolge una funzione analoga al tool precedente, ma applicata alle notizie. A partire da un argomento o dal nome di un videogioco, verifica se siano già presenti news correlate nel database, analizzando titolo, overview e relazioni con il nodo Game. L'obiettivo è prevenire la pubblicazione di notizie ridondanti.
- `get_unreviewed_released_games_tool`: Questo tool viene utilizzato durante la fase di pianificazione editoriale o di suggerimento per recuperare l'elenco dei videogiochi scoperti durante la stesura degli altri post, che sono già usciti sul mercato ma che non sono ancora stati oggetto di una recensione sul blog. Il tool esegue quindi una query sul grafo selezionando tutti i nodi Game con attributo `uscito` impostato a `true` e verificando che non esista alcuna recensione collegata tramite la relazione `ABOUT`. L'insieme dei risultati costituisce un backlog di giochi già disponibili sul mercato che possono essere proposti per la generazione di nuove recensioni.
- `cerca_post_gioco`: Permette di ricercare all'interno del database tutti i post (news o recensioni) relativi a un determinato videogioco o a un'intera serie. La ricerca viene eseguita sui campi titolo e overview dei nodi Post, utilizzando il nome del gioco e, facoltativamente, quello della serie. I risultati vengono ordinati cronologicamente e restituiti in formato JSON, consentendo all'agente di recuperare rapidamente lo storico dei contenuti già pubblicati.

4 Prompt ed Altri File

I prompt utilizzati per guidare il comportamento dei Large Language Model sono raccolti all'interno della cartella `prompts` e organizzati in file distinti in base alla funzionalità svolta. Ogni file contiene le istruzioni specifiche utilizzate dai diversi nodi del sistema per orientare il ragionamento e la generazione del testo. Ad esempio, nel file `prompt_merge.py` sono definiti i prompt utilizzati dal nodo incaricato della stesura della bozza del post; in base alla tipologia di contenuto da produrre (recensione o news), viene selezionato un prompt differente così da adattare lo stile e la struttura del testo al contesto. Analogamente, il file `prompt_suggestion.py` contiene i prompt utilizzati dal sottografo dedicato alla generazione di suggerimenti editoriali, definendo le regole di ragionamento che l'agente deve seguire per proporre nuovi contenuti.

Un ruolo fondamentale nell'architettura è svolto dal file `State_template.py`, situato nella directory principale del progetto. Questo file contiene:

- Gli *state* utilizzati dal grafo principale e dai vari sottografi, i quali fungono da memoria a breve termine per il sistema, archiviando le informazioni relative al flusso esecutivo e garantendo il corretto scambio di dati tra i nodi durante l'esecuzione del flusso di lavoro.
- Gli schemi di output strutturato, realizzati mediante modelli Pydantic, che consentono di vincolare il formato delle risposte generate dagli LLM.
- Le configurazioni degli strumenti utilizzati dai vari nodi, tra cui l'inizializzazione dei

diversi modelli linguistici utilizzati nelle varie fasi del workflow (`llm`, `llm_p`, `llm_x` e `llm_merge`) e del client Tavily per le ricerche sul web.

5 MCP (Model Context Protocol)

Per rendere il sistema modulare ed estensibile è stato adottato il **Model Context Protocol (MCP)**, un protocollo standard che consente agli LLM di interagire con strumenti e servizi esterni attraverso un'interfaccia uniforme. L'obiettivo di MCP è separare la logica dell'agente dall'implementazione dei singoli strumenti, permettendo di aggiungere o sostituire servizi senza modificare il funzionamento interno dell'agente.

Nel progetto è stato sviluppato un server MCP dedicato (presente all'interno della cartella `mcp_server` nel file `search_server.py`) utilizzando la libreria FastMCP, la quale espone come servizio il tool di ricerca web basato su Tavily, reso disponibile attraverso il protocollo MCP utilizzando il trasporto `streamable-http`. Il decoratore `@mcp_server.tool()` è fondamentale in quanto permette di esporre la funzione `tavily_search_tool` come servizio accessibile tramite il protocollo MCP.

Dal lato dell'agente è stato implementato un client basato sulla libreria `langchain-mcp-adapters` (all'interno della cartella `tools` nel file `tavily_search_tool.py`), il quale gestisce l'intera comunicazione con il server MCP. Prima di ogni richiesta viene verificata la disponibilità del server, qualora questo non risulti attivo, il client provvede automaticamente ad avviarlo come processo separato e attende che il servizio diventi raggiungibile prima di procedere con l'invocazione del tool.

Una volta stabilita la connessione, il client recupera dinamicamente gli strumenti esposti dal server e invoca quello necessario passando i parametri richiesti. La risposta ricevuta viene quindi elaborata e convertita in un formato testuale facilmente utilizzabile dall'agente del sistema. Per garantire la robustezza dell'applicazione, la comunicazione con il server MCP viene eseguita all'interno di un thread dedicato e protetta mediante timeout configurabili. In questo modo eventuali ritardi, indisponibilità del server o errori di comunicazione non compromettono il funzionamento dell'intero workflow: le eccezioni vengono intercettate e il sistema prosegue l'esecuzione informando l'agente dell'impossibilità di effettuare la ricerca web.

L'adozione di questo protocollo offre numerosi vantaggi dal punto di vista architetturale. In particolare, consente di disaccoppiare completamente la logica degli agenti dai servizi esterni, migliorando la manutenibilità del sistema e favorendo la riusabilità dei componenti. Inoltre, l'utilizzo di un'interfaccia standardizzata rende l'architettura facilmente estendibile: nuovi strumenti, API o servizi potranno essere integrati semplicemente esponendoli tramite il server MCP, senza richiedere modifiche alla logica decisionale degli agenti.

6 Knowledge Graph (Neo4j)

Introduzione

Nell'agente sviluppato, è stato utilizzato un Knowledge Graph per rappresentare e gestire in maniera strutturata le informazioni relative ai contenuti editoriali pubblicati. Questo approccio consente di modellare il dominio come una rete di entità interconnesse, in cui

ogni elemento informativo non è isolato ma è inserito all'interno di un contesto relazionale esplicito.

La struttura del grafo è basata su due componenti fondamentali, i **nodi** e le **relazioni**. I nodi rappresentano le entità del dominio, come videogiochi, post editoriali, case editrici, generi e fonti informative. Ogni nodo può contenere proprietà descrittive che ne arricchiscono il contenuto semantico, ad esempio il nome del gioco, lo stato di uscita o il contenuto testuale di un articolo. Le relazioni, invece, modellano i collegamenti tra le entità e sono tipizzate per rappresentare il significato del legame, come ad esempio "un post tratta un determinato gioco", "un contenuto menziona un gioco" oppure "un gioco appartiene a un genere specifico".

Per l'implementazione è stato utilizzato il database **Neo4j**, scelto per la sua natura nativa orientata ai grafi e per l'efficienza nell'esecuzione di query su relazioni complesse. Attraverso il linguaggio **Cypher** vengono creati e aggiornati i nodi e le relazioni del grafo in base ai contenuti generati dal sistema.

Rispetto ai database relazionali (SQL) i Knowledge Graph risultano più adatti quando le informazioni sono fortemente connesse. Nei database relazionali i dati vengono organizzati in tabelle e le relazioni sono gestite tramite chiavi esterne e operazioni di join. Questo approccio è molto efficace per dati strutturati e schemi stabili, ma diventa meno efficiente quando le relazioni tra entità sono numerose e profondamente annidate. Un'altra alternativa è rappresentata dai database vettoriali, che memorizzano le informazioni sotto forma di embedding numerici e consentono la ricerca basata sulla similarità semantica. Questa tipologia è particolarmente utile per recuperare contenuti concettualmente simili, ma non consente di rappresentare esplicitamente le relazioni tra entità.

Implementazione ed Utilizzo del Knowledge Graph in Neo4j

Nel progetto, il Knowledge Graph è stato implementato utilizzando Neo4j come database a grafo per garantire la persistenza strutturata dei contenuti editoriali generati dal sistema. Ogni post prodotto dall'agente viene trasformato in una rappresentazione semantica arricchita, in cui le informazioni testuali vengono estratte automaticamente e convertite in entità e relazioni.

Le entità principali del grafo sono i nodi `Post`, che rappresentano e modellano ogni contenuto editoriale pubblicato dal sistema (sia news che recensioni). Ogni nodo Post contiene informazioni descrittive fondamentali come identificativo univoco, titolo, tipo di contenuto (News o Review), data di pubblicazione, contenuto completo e una sintesi semantica (`overview`).

Attorno a questo nodo centrale vengono costruite diverse tipologie di entità collegate tramite relazioni semantiche:

- **Game**: rappresenta i videogiochi coinvolti o citati nei contenuti. Ogni Post è collegato a uno o più nodi Game tramite una relazione che indica il gioco principale trattato (`ABOUT`) e i giochi secondari citati all'interno del contenuto (`MENTIONS`). I nodi Game includono anche un attributo che indica se il titolo è già stato pubblicato o è ancora in fase di rilascio, informazione fondamentale per il suggerimento editoriale, il quale non deve suggerire la recensione di giochi ancora non rilasciati.
- **Genere**: i generi videoludici vengono modellati come nodi indipendenti riutilizza-

bili. Ogni Post e ogni Game possono essere collegati a uno o più generi tramite relazioni dedicate. La scelta di collegare sia il Post che il gioco (Game) al genere permette di scoprire il genere di un gioco direttamente dal nodo Game.

- **Publisher**: le case editrici e software house vengono rappresentate come entità riutilizzabili del grafo. Questa modellazione permette di analizzare facilmente la produzione editoriale per singolo publisher e di stabilire connessioni tra giochi e contenuti pubblicati dallo stesso attore. I publisher vengono legati tramite la relazione `PUBLISHED_BY` sia alle entità Game che ai nodi Post.
- **Source**: le fonti web utilizzate per la generazione dei contenuti vengono normalizzate e salvate come nodi separati. Ogni Post è collegato alle proprie fonti tramite relazioni `SOURCED_FROM` che conservano l'URL originale, consentendo la tracciabilità delle informazioni utilizzate per scrivere il post anche dopo la pubblicazione.



Figura 6: Visualizzazione delle entità create a seguito di una recensione.

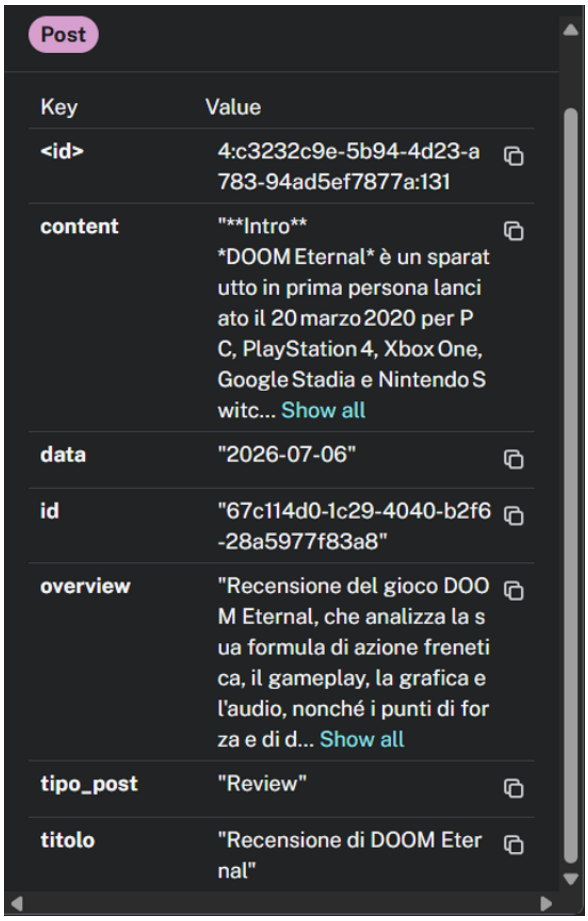


Figura 7: Proprietà di un nodo Post di tipo recensione.

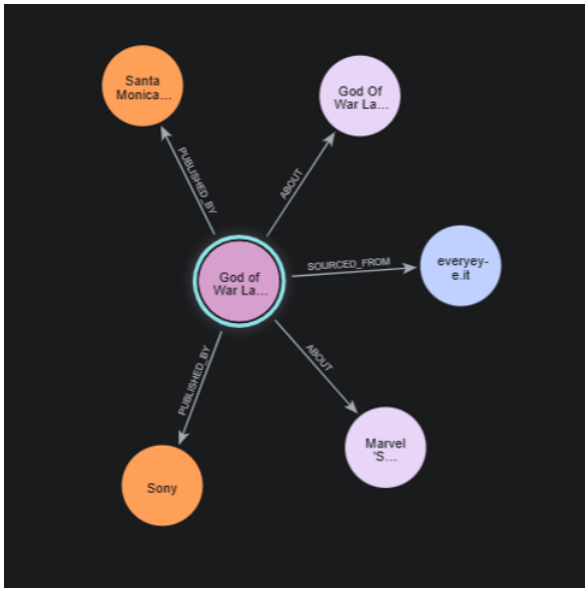


Figura 8: Visualizzazione delle entità create a seguito di una news.

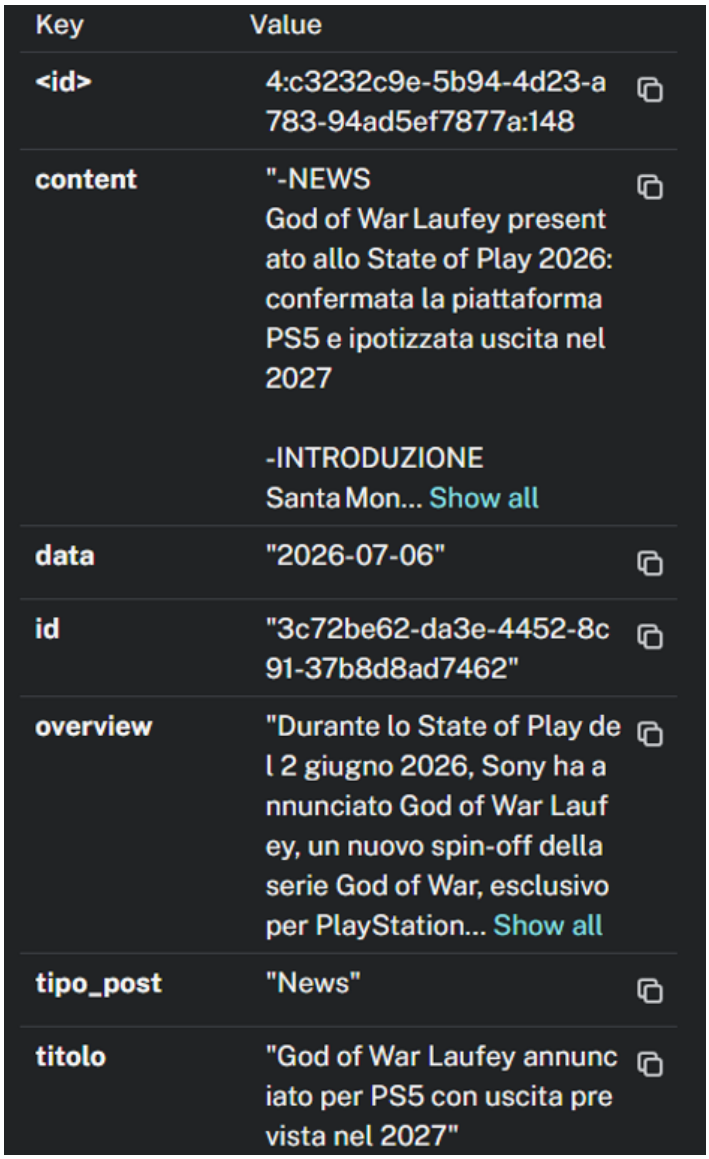


Figura 9: Proprietà di un nodo Post di tipo News.

il Knowledge Graph Neo4j non viene utilizzato soltanto come database di storage dei contenuti pubblicati, ma assume un ruolo centrale anche come memoria dello stato della pianificazione editoriale.

In particolare, la pianificazione non è mantenuta in memoria temporanea, ma viene modellata come una coda strutturata di nodi `Pianificazione`, ciascuno dei quali rappresenta una singola unità di lavoro editoriale, in modo da poter accedere alla pianificazione effettuata in un qualunque momento (anche giorni dopo). Ogni nodo contiene proprietà come titolo, ordine e stato, che permettono di definire sia il contenuto del suggerimento sia la sua posizione all'interno del flusso di pianificazione.

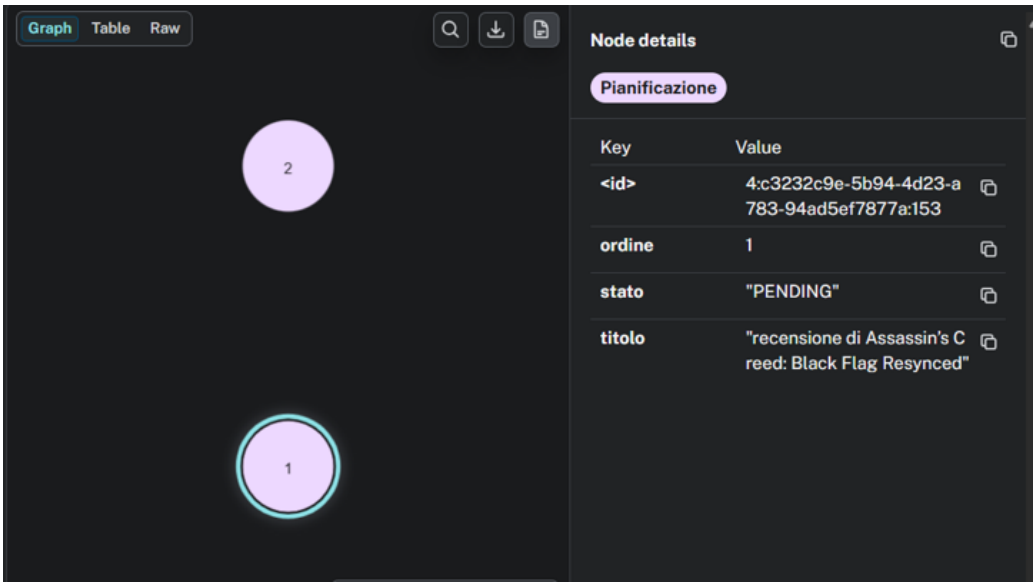


Figura 10: Visualizzazione dei nodi per la pianificazione editoriale.

La Figura 10 riporta due nodi creati a seguito dell'accettazione della pianificazione proposta. In particolare i nodi prevedono uno stato, il quale sarà `PENDING`, il titolo, che conterrà il suggerimento editoriale per il post, ed un campo ordine, il quale serve a tenere traccia dell'ordine con cui i post sono stati suggeriti all'interno della pianificazione, in modo da recuperarli seguendo l'ordine editoriale proposto.

Limiti Riscontrati

Passando ai limiti del knowledge graph, uno degli aspetti più delicati riguarda la costruzione e il mantenimento. Sebbene l'utilizzo di un LLM per l'estrazione automatica delle informazioni permette di popolare il grafo senza intervento umano, la qualità della struttura risultante dipende fortemente dalla correttezza dei dati estratti dal modello. In particolare, la creazione dei nodi relativi a videogiochi, publisher, generi e fonti può risentire di piccole variazioni nella denominazione delle entità. Ad esempio, uno stesso genere videoludico può essere indicato come *FPS*, *First Person Shooter* oppure semplicemente *Sparatutto*; analogamente possono verificarsi differenze come *Action RPG* e *Action-RPG*. In assenza di una normalizzazione semantica completa, il sistema tende a considerare queste denominazioni come entità distinte, generando nodi duplicati che rappresentano in realtà lo stesso concetto.

Un problema analogo riguarda i titoli dei videogiochi. Differenze minime nella scrittura, come la presenza o meno dell'apostrofo, caratteri tipografici differenti (ad esempio apostrofo semplice `'` e apostrofo tipografico `’`), abbreviazioni o titoli incompleti possono portare alla creazione di nodi separati. Per mitigare questi problemi sono state adottate alcune strategie di normalizzazione, come la rimozione degli spazi superflui, la standardizzazione dei nomi e l'utilizzo delle operazioni `MERGE` di Neo4j, che evitano la duplicazione di nodi con lo stesso identificativo testuale. Tuttavia, queste tecniche operano esclusivamente a livello lessicale e non sono sufficienti a riconoscere automaticamente sinonimi, abbreviazioni o varianti ortografiche dello stesso concetto.

7 RAG (Retrieval-Augmented Generation)

Introduzione

Il Retrieval-Augmented Generation (RAG) è una tecnica che combina un modello linguistico (LLM) con il recupero di documenti esterni. L'obiettivo principale è superare uno dei limiti intrinseci degli LLM, ovvero il fatto che generano le risposte basandosi esclusivamente sulle conoscenze acquisite durante la fase di addestramento. Di conseguenza, quando il modello non dispone di informazioni sufficienti su un determinato argomento, può produrre risposte apparentemente plausibili ma non necessariamente corrette, fenomeno noto come *allucinazione*.

Nel sistema sviluppato, il RAG viene impiegato per integrare il processo di recupero delle informazioni necessarie alla stesura della bozza relativa all'argomento richiesto dall'utente. In particolare, le informazioni ottenute tramite ricerche sul web vengono arricchite con contenuti provenienti da una base documentale ritenuta affidabile, costituita da recensioni, informazioni su eventi, case editrici e articoli precedentemente revisionati.

Il funzionamento del RAG può essere suddiviso in due fasi principali:

La prima è la **fase di indicizzazione**. I documenti vengono acquisiti, suddivisi in porzioni di testo (*chunk*) e successivamente convertiti in embedding, ossia rappresentazioni vettoriali che catturano il significato semantico del testo. Gli embedding vengono quindi memorizzati all'interno di un database vettoriale, che nel sistema sviluppato è **ChromaDB**, consentendo di effettuare ricerche basate sulla similarità semantica anziché sulla semplice corrispondenza tra parole chiave.

La seconda fase riguarda il **recupero delle informazioni e la generazione della risposta**. Quando il sistema deve reperire informazioni per la stesura della bozza e decide di interrogare il RAG, la query viene trasformata in un embedding utilizzando lo stesso modello impiegato durante l'indicizzazione. Il database vettoriale confronta quindi tale vettore con quelli memorizzati e restituisce i documenti semanticamente più simili alla richiesta. I contenuti recuperati vengono successivamente forniti al modello linguistico insieme alle informazioni ottenute tramite le ricerche sul web, permettendogli di generare il testo finale utilizzando entrambe le fonti di conoscenza.

Creazione della Base di Conoscenza

La costruzione della base documentale del RAG rappresenta una fase di preprocessing che viene eseguita prima dell'utilizzo del sistema. Nel progetto sviluppato, la base documentale è costituita da recensioni, informazioni generali su eventi, case editrici ed articoli in formato Markdown.

Durante la fase di indicizzazione, ogni documento viene suddiviso nelle sue diverse sezioni (ad esempio gameplay, trama, comparto tecnico o conclusioni), mantenendo i metadati che identificano il videogioco e la sezione di appartenenza. Successivamente, ogni sezione viene ulteriormente suddivisa in frammenti di dimensione controllata (chunk). La suddivisione viene effettuata introducendo una sovrapposizione tra chunk consecutivi, così da evitare che informazioni relative alla stessa sezione vengano separate artificialmente troncando a metà le frasi.

Una volta ottenuti i chunk, ciascuno di essi viene trasformato in un embedding. Gli

embedding vengono quindi memorizzati all'interno del database vettoriale ChromaDB, che costituisce la base di conoscenza interrogabile dal sistema.

Utilizzo del RAG

Durante la ricerca delle informazioni per poter scrivere la bozza, l'agente può decidere di interrogare il RAG attraverso il tool `RAG_tool` quando ritiene necessario recuperare informazioni approfondite e corrette già presenti nella memoria del sistema. Quando il tool viene invocato, la query prodotta dall'agente viene convertita nello stesso spazio vettoriale utilizzato durante l'indicizzazione. Il database Chroma confronta l'embedding della query con quelli memorizzati e restituisce i frammenti semanticamente più vicini alla richiesta. Per aumentare l'affidabilità delle informazioni recuperate, il sistema applica una soglia minima di similarità: vengono utilizzati soltanto i documenti che superano un determinato livello di pertinenza, mentre i risultati considerati troppo distanti semanticamente vengono scartati.

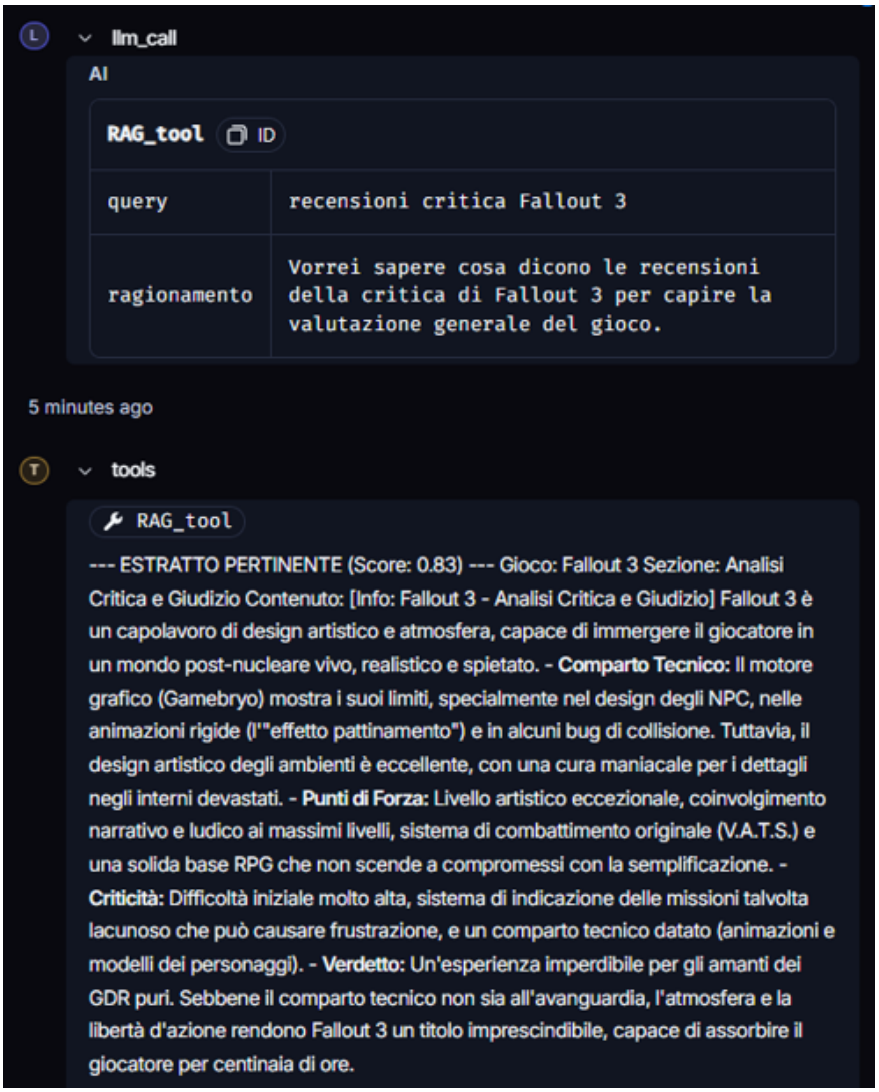


Figura 11: Recupero di informazioni dal RAG.

Limiti Riscontrati

Sebbene l'architettura RAG si sia dimostrata efficace nel migliorare la qualità e l'affidabilità delle risposte, emergono alcune limitazioni strutturali intrinseche alla ricerca semantica. La criticità principale risiede nella gestione delle ambiguità semantiche: a fronte di una query come "informazioni relative al gameplay di Far Cry 3", il sistema tende a dare un peso maggiore al concetto generale di "gameplay" rispetto all'entità specifica del gioco "Far Cry 3". Per mitigare questa problematica, è stata introdotta una soglia di similarità minima volta a filtrare i risultati, e si è optato per un approccio che delega la validazione della pertinenza direttamente al LLM, il quale agisce come un filtro cognitivo avanzato.

Un ulteriore limite strutturale emerso riguarda il tentativo di estendere il processo RAG sfruttando il Knowledge Graph come fonte di espansione della query. L'idea iniziale consisteva nell'utilizzare le relazioni presenti nel grafo per individuare informazioni non ricavabili direttamente dalla richiesta dell'utente. Tuttavia, nella pratica questo approccio non ha prodotto un miglioramento significativo della qualità del retrieval, poiché molte delle relazioni presenti nel grafo risultano semanticamente corrette, ma poco rilevanti rispetto all'obiettivo della generazione del contenuto.

Un ulteriore approccio sperimentato ha riguardato l'utilizzo diretto delle informazioni recuperate dal Knowledge Graph per arricchire la stesura delle recensioni attraverso confronti con altri titoli collegati. Tuttavia, anche questa strategia ha evidenziato diversi limiti: i confronti risultavano spesso poco utili e poco professionali, e il sistema non è in grado di stabilire autonomamente se due titoli siano direttamente confrontabili. Oltre a questi problemi è presente un enorme limite rappresentato dalla context window del modello. Per questo motivo si è deciso di non utilizzare il Knowledge Graph per generare automaticamente confronti all'interno delle recensioni, impiegando le informazioni memorizzate nel grafo esclusivamente per proporre all'utente contenuti correlati pubblicati sul blog.

8 Esecuzione Agente

In questo capitolo vengono presentati alcuni esempi di esecuzione dell'agente, con l'obiettivo di illustrare i principali flussi operativi implementati nel sistema. In particolare, saranno analizzati i seguenti casi d'uso:

1. Richiesta da parte dell'editore del blog di generare una recensione di un videogioco (Recensione di God of War Ragnarök).
2. Richiesta da parte dell'editore del blog di generare una news relativa a un videogioco (News relativa all'annuncio del nuovo God of War Laufey).
3. Richiesta di un consiglio relativo a un videogioco.
4. Richiesta di generazione di una pianificazione editoriale della durata di tre giorni.

L'esecuzione del workflow viene effettuata mediante **LangGraph Studio**, che consente di visualizzare in tempo reale il grafo, i nodi attraversati, lo stato condiviso e il flusso di esecuzione dell'agente. L'interazione con l'utente, necessaria per approvare, rifiutare o richiedere modifiche alle bozze generate, avviene invece attraverso la piattaforma **Agent Inbox**.

8.1 Flusso Esecutivo per la Scrittura di una Recensione/News

Di seguito viene illustrato il flusso esecutivo relativo alla generazione di una recensione, il cui comportamento risulta sostanzialmente analogo a quello adottato per la generazione di una news, tranne per l'utilizzo dei tool `ottieni_prezzo_gioco` e `recensioni_utenti_steam`, i quali non vengono utilizzati per la scrittura di una news. L'analisi dettagliata verrà presentata esclusivamente per il caso della richiesta "Recensione di God of War Ragnarök".

Esecuzione e Invocazione Tool

La Figura 12 mostra la sequenza di invocazione dei tool durante il recupero delle informazioni necessarie per la scrittura della recensione. Dopo aver ricevuto la richiesta dell'utente, l'agente analizza progressivamente quali informazioni risultano necessarie e chiama i tool disponibili in maniera iterativa, fino a quando ritiene di aver raccolto un contesto informativo sufficiente per procedere alla stesura della bozza.

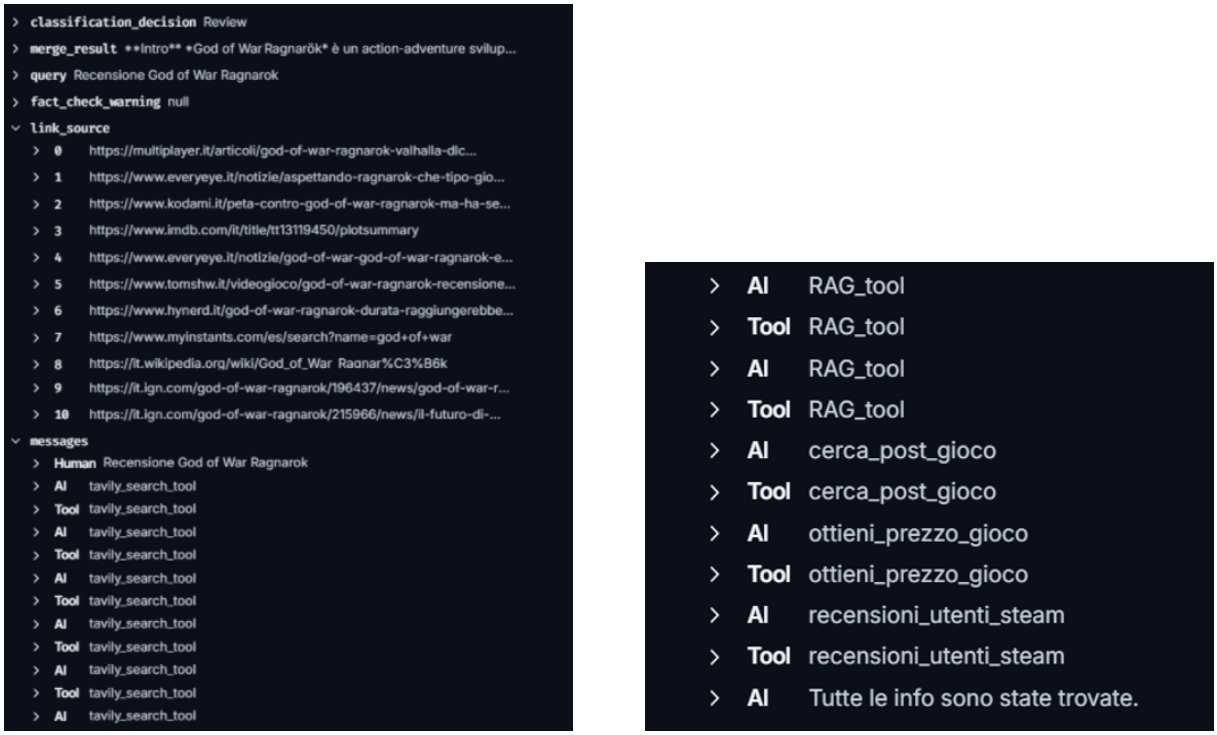


Figura 12: Chiamata ai tool per la recensione.

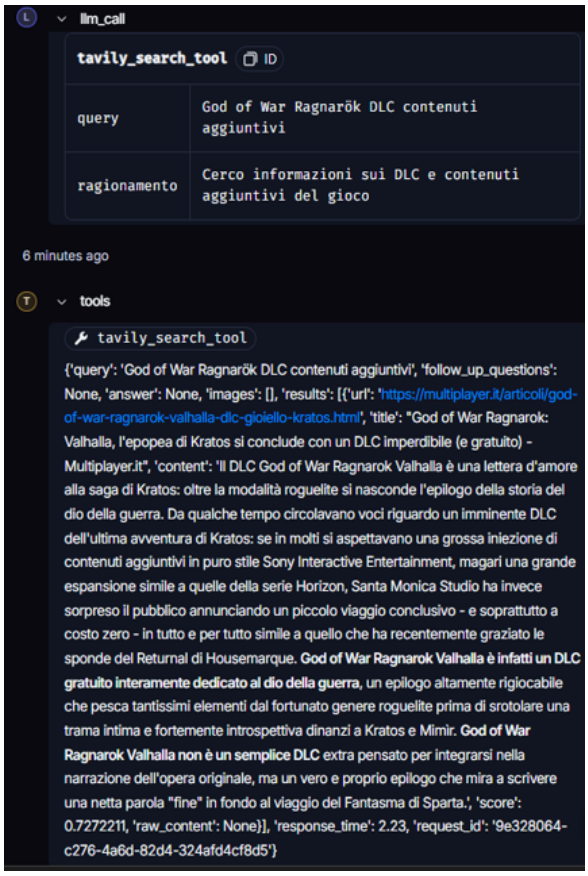


Figura 13: Esempio di chiamata al tool Tavily.

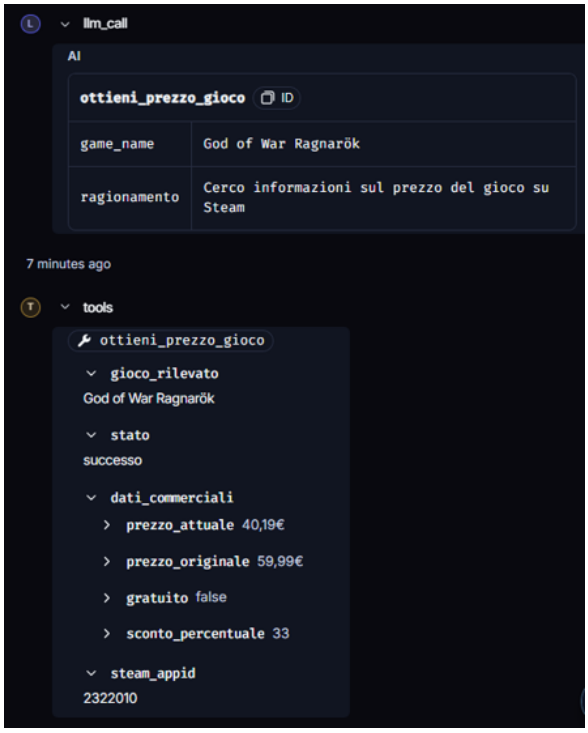


Figura 14: Esempio di chiamata al tool `ottieni_prezzo_gioco`.

A seguire saranno riportate l'elenco delle invocazioni dei tool effettuate per la ricerca di informazioni per la News e qualche dettaglio di chiamata al singolo tool.

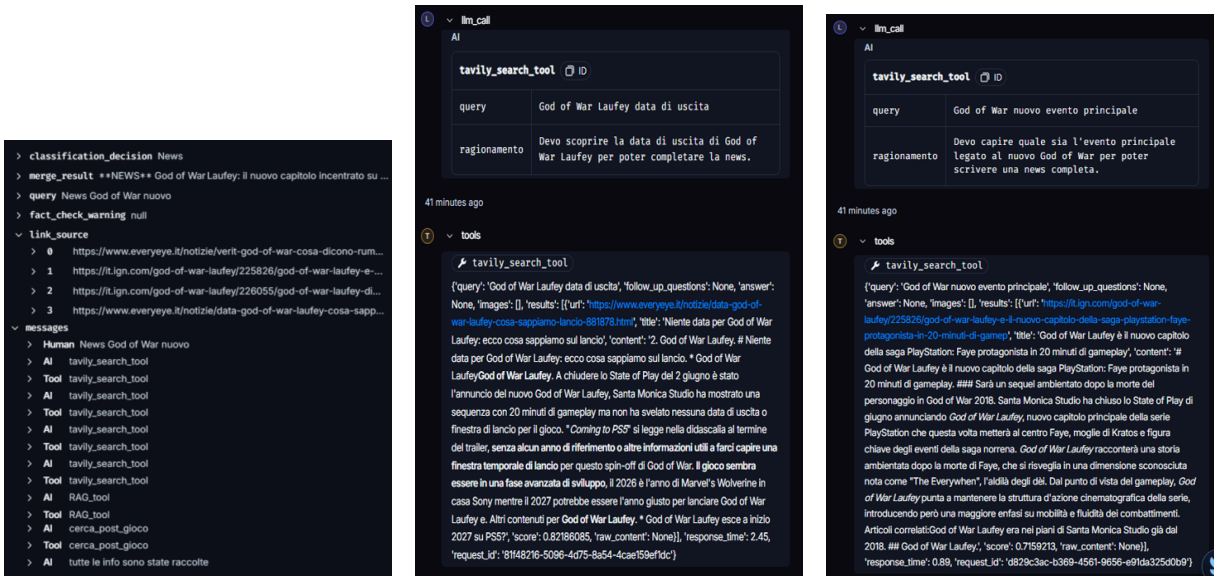


Figura 15: Invocazione dei tool e dettagli delle chiamate Tavily per la News.

Bozza Proposta

Di seguito vengono riportate le bozze della recensione e della news sviluppate dal nodo `merge_result` nelle due esecuzioni, le quali vengono sottoposte alla revisione umana. L'utente può decidere di modificare manualmente la bozza, chiedere all'agente di apportare delle modifiche o semplicemente accettarla o rifiutarla.

Recensione:

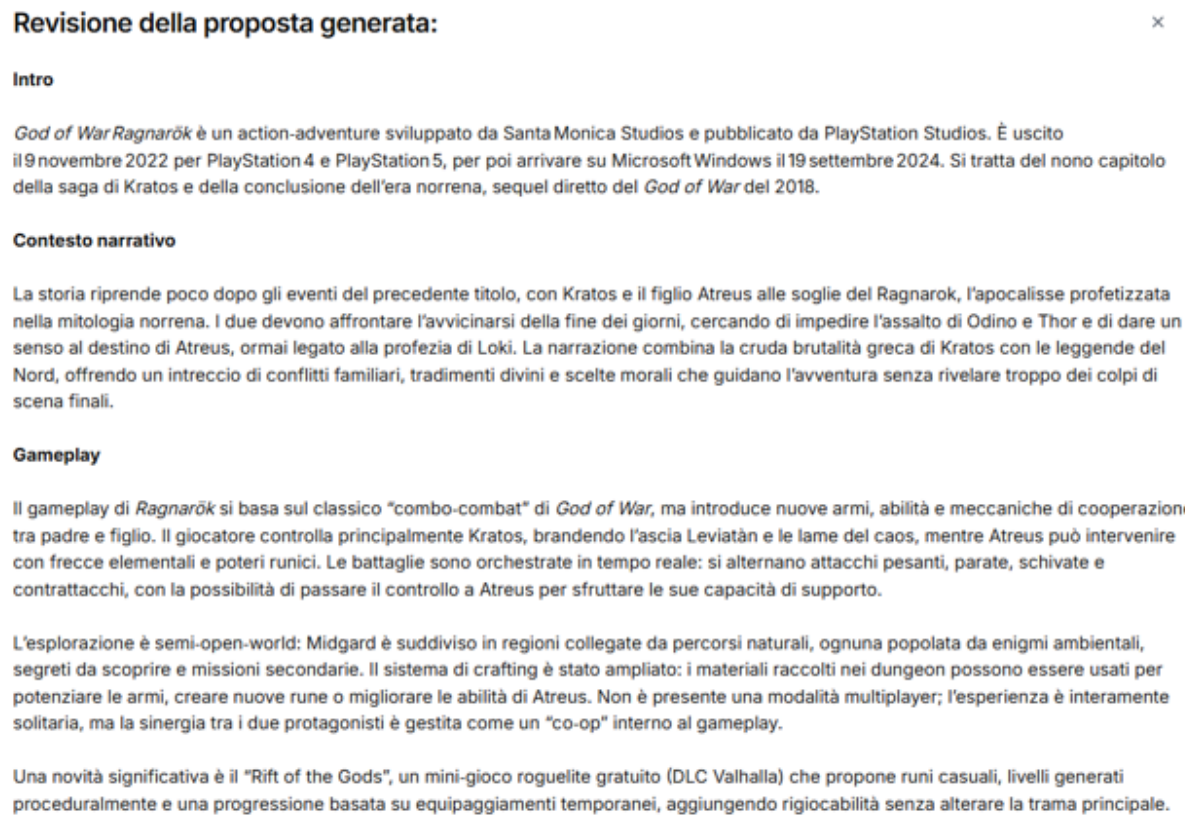


Figura 16: Stato dell'agente durante la generazione della recensione.

Grafica, direzione artistica e audio

Grazie al motore proprietario di Santa Monica, il titolo sfrutta al massimo le capacità della PS5 e delle GPU di fascia alta su PC, offrendo ambienti ricchi di dettagli, effetti di luce dinamici e texture ad alta risoluzione. La resa visiva dei paesaggi norreni, dalle foreste gelide alle rovine incandescenti, è spettacolare, mentre le animazioni dei personaggi mantengono un alto livello di realismo. La colonna sonora, composta da Bear McCreary, combina cori epici, percussioni tribali e temi orchestrali che amplificano l'atmosfera mitologica. Gli effetti sonori – dal clangore delle armi al rombo dei fulmini di Thor – sono curati nei minimi particolari, contribuendo a un'immersione totale.

Punti di forza e difetti

Pro (basati sulle recensioni della community di Steam):

- Un'esperienza narrativa avvincente che conclude in modo soddisfacente la saga norrena.
- Combattimento fluido e profondo, con nuove armi e abilità che mantengono alta la tensione.
- Grafica di altissimo livello e colonna sonora epica.
- DLC gratuito "Valhalla" che aggiunge contenuti roguelite rigiocabili.

Contro (sempre dalle opinioni degli utenti):

- Alcuni giocatori segnalano crash ricorrenti, soprattutto su configurazioni PC non ottimizzate, con un'interruzione ogni due ore di gioco.
- Problemi di stabilità e ottimizzazione su sistemi di fascia alta, con occasionali cali di frame.
- Per una parte della community il ritmo iniziale può apparire lento e la struttura delle missioni ripetitiva rispetto a titoli concorrenti come *Shadow of War* o *Assassin's Creed Valhalla*.

Il prezzo attuale su Steam è di **40,19 €**, scontato del 33 % rispetto al listino originale di 59,99 €, un valore competitivo per un titolo di tale portata.

Figura 17: Bozza della recensione di God of War Ragnarök generata dal nodo `merge_result`.

DLC e contenuti aggiuntivi

Il gioco ha ricevuto il DLC gratuito *God of War Ragnarök Valhalla*, un epilogo roguelite che espande la narrazione con nuove missioni, meccaniche di crafting temporaneo e una modalità di gioco più "arcade". Non sono previsti ulteriori pacchetti a pagamento al momento.

Suggerimenti e collegamenti editoriali

Per chi è interessato a proseguire l'avventura di Kratos, è già stata annunciata la prossima iterazione della serie, *God of War Laufey*, che si concentrerà sulla figura di Faye e promette un combattimento ancora più fluido. Tenete d'occhio i prossimi articoli della nostra rubrica per approfondimenti su questo futuro capitolo e sulle potenziali evoluzioni della saga.

Nota: i pro e i contro riportati nella sezione 5 sono stati ricavati dai risultati RAG (interni) relativi alle recensioni della community di Steam.

Figura 18: Interfaccia di revisione della bozza — sezione Punti di forza e DLC.

News:

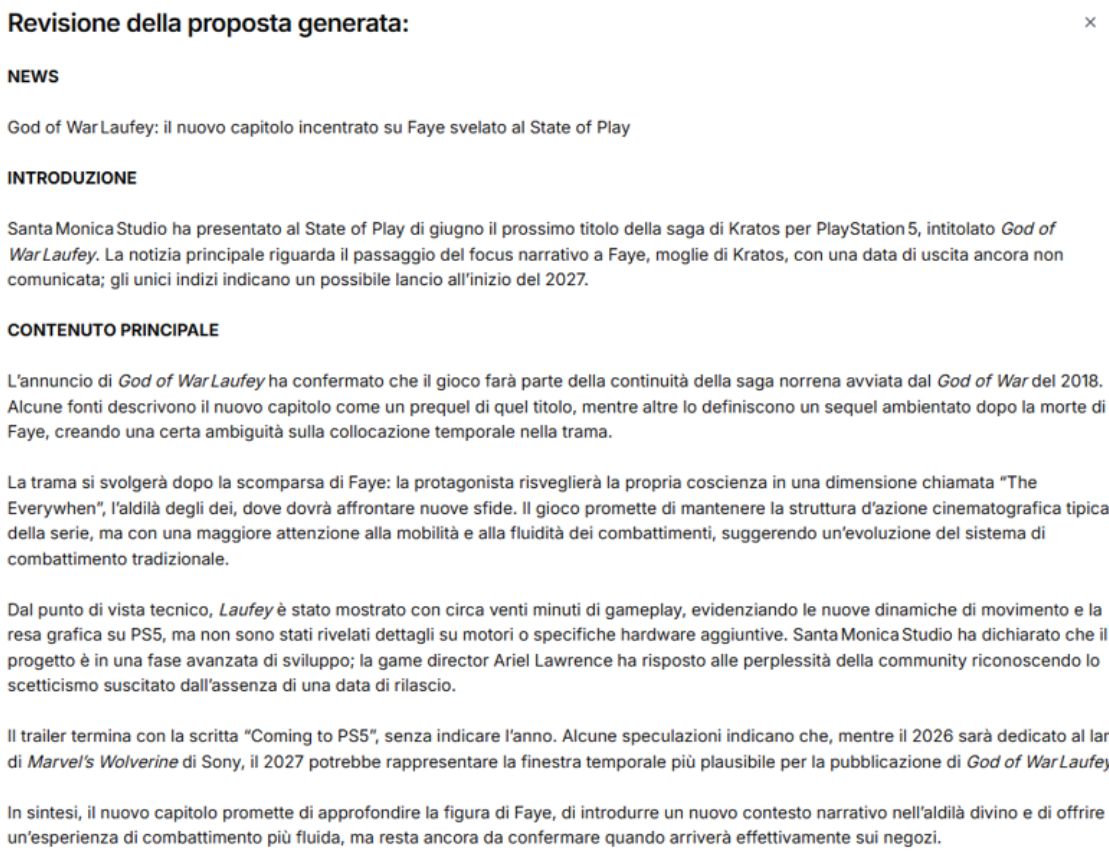


Figura 19: Bozza della news relativa a God of War Laufey generata dal nodo `merge_result`.

Nodo del Post

A seguito dell'approvazione tramite HITL della bozza, viene creato il nodo di tipo Post che modella la Recensione/News appena approvata con tutte le relative informazioni. Di seguito sono riportate le entità create a seguito dell'approvazione della recensione e della news.

Recensione:

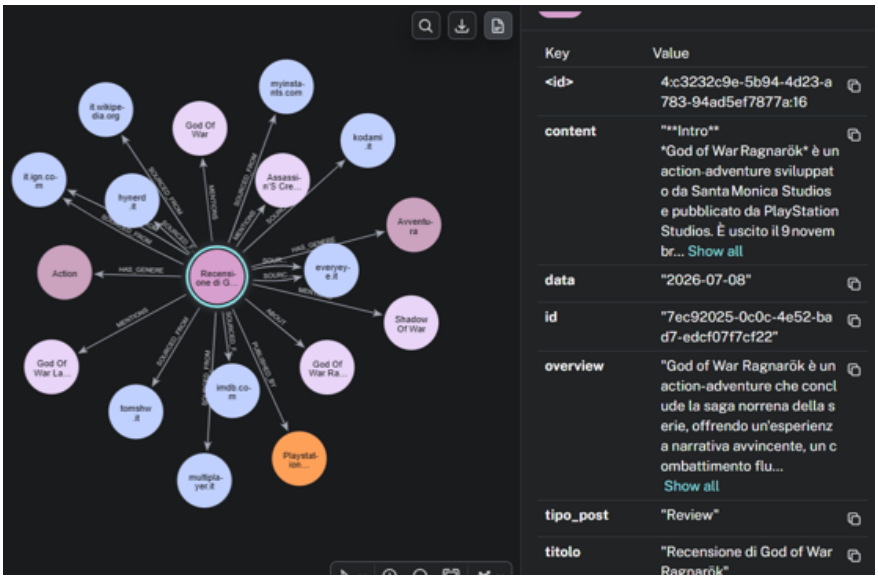


Figura 20: Entità create nel Knowledge Graph a seguito dell'approvazione della recensione.

News:

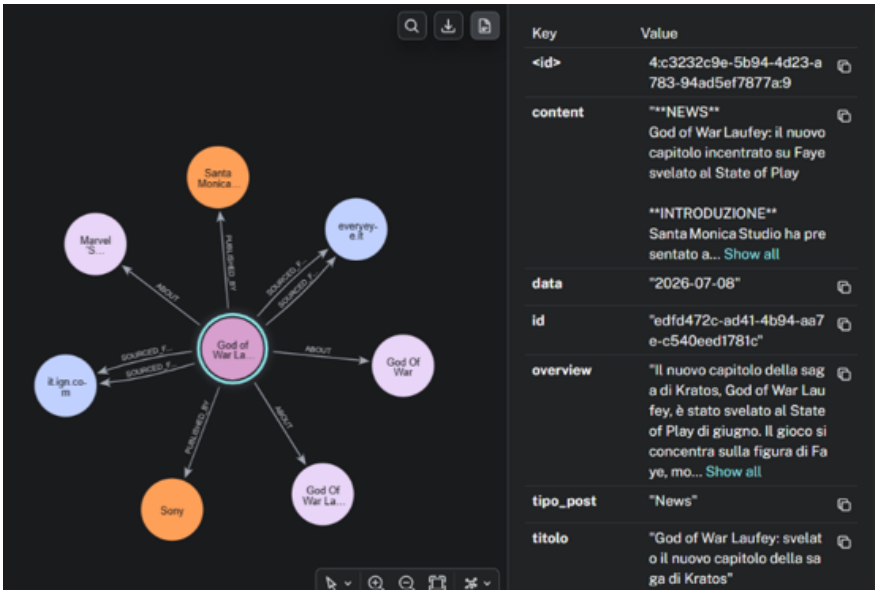


Figura 21: Entità create nel Knowledge Graph a seguito dell'approvazione della news.

8.2 Flusso Esecutivo per il Suggerimento Editoriale

In questo paragrafo verrà visualizzato il flusso esecutivo, i tool invocati ed il ragionamento applicato dal modello per suggerire un post da poter pubblicare. Quando l'utente chiede un suggerimento su cosa poter pubblicare la richiesta viene gestita dal `suggestion_subgraph`, il quale ha l'obbligo di esaminare i post già pubblicati e suggerirne uno in modo da bilanciare il contenuto editoriale.

Esecuzione e Invocazione Tool

Nella Figura 22 sono riportate le chiamate effettuate dal LLM per recuperare tutte le informazioni necessarie per fornire un suggerimento che bilanci il contenuto. In particolare, come primo tool viene invocato `get_planning_context_tool` il quale restituisce le news più recenti pubblicate, le recensioni più recenti pubblicate, e le varietà tematiche ossia le tipologie di giochi recensiti. Sulla base di tali informazioni il modello baserà il suo ragionamento per scegliere la tipologia di post da proporre (News/Review) ed in caso di scelta della recensione anche il genere del gioco. Se la scelta ricade su una recensione allora verrà chiamato il tool `get_unreviewed_released_games` per vedere se ci sono giochi precedentemente scoperti in attesa di recensione.

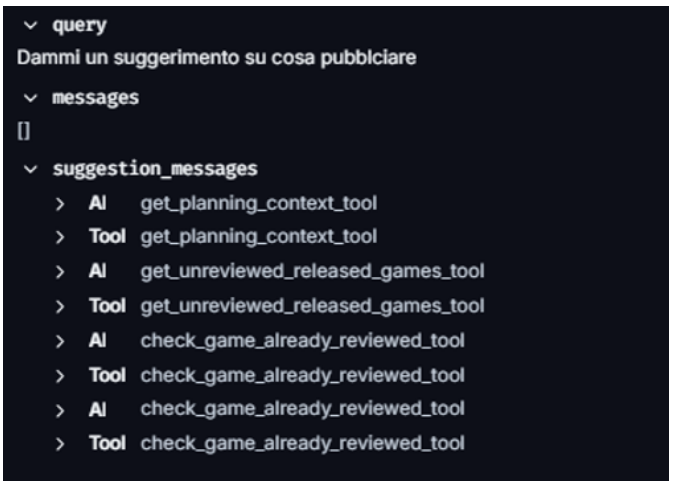


Figura 22: Invocazione dei tool per il suggerimento editoriale.

Una volta che l'LLM ha il suggerimento da fornire all'utente terminerà l'esecuzione fornendo il ragionamento seguito e sottoporrà il consiglio all'approvazione tramite HITL all'utente.

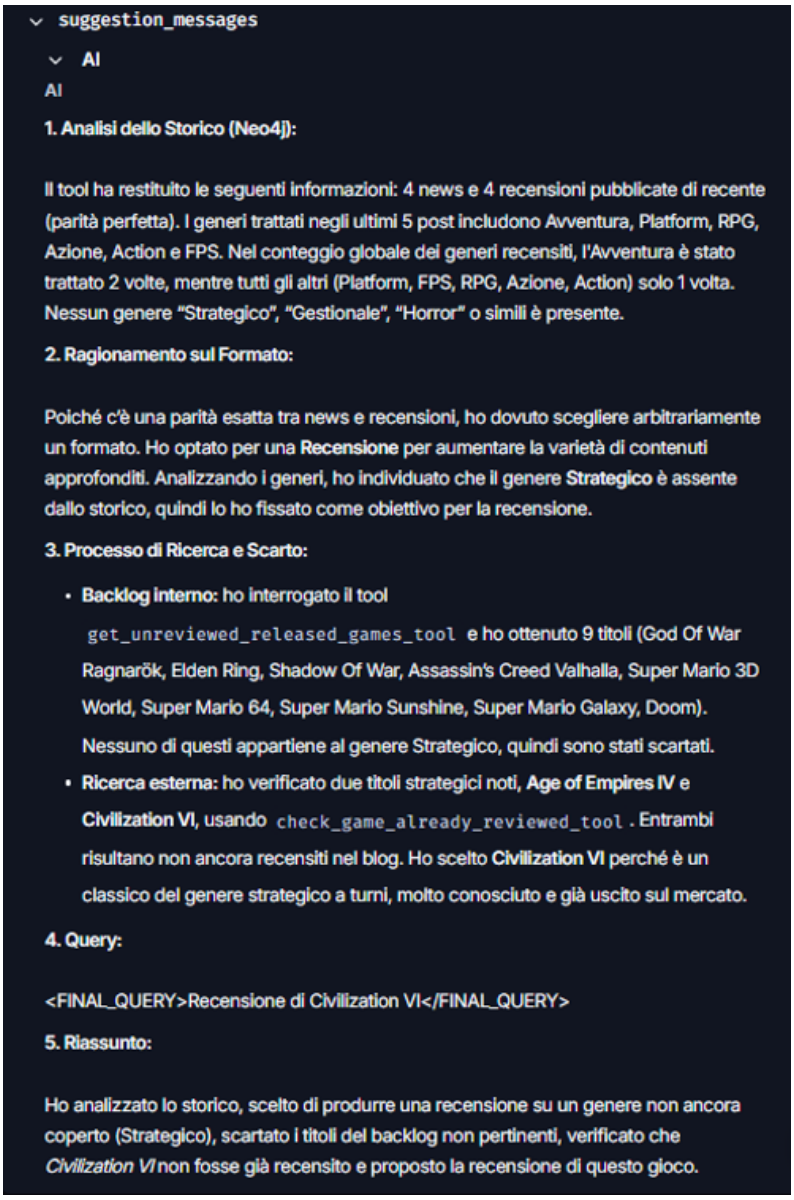


Figura 23: Ragionamento del nodo di suggerimento.

Per completezza di seguito viene riportato un altro caso di richiesta di un suggerimento quando le tipologie delle pubblicazioni nel blog non sono bilanciate, ma sbilanciate, come in questo caso in cui il blog prevede molto più recensioni che news.

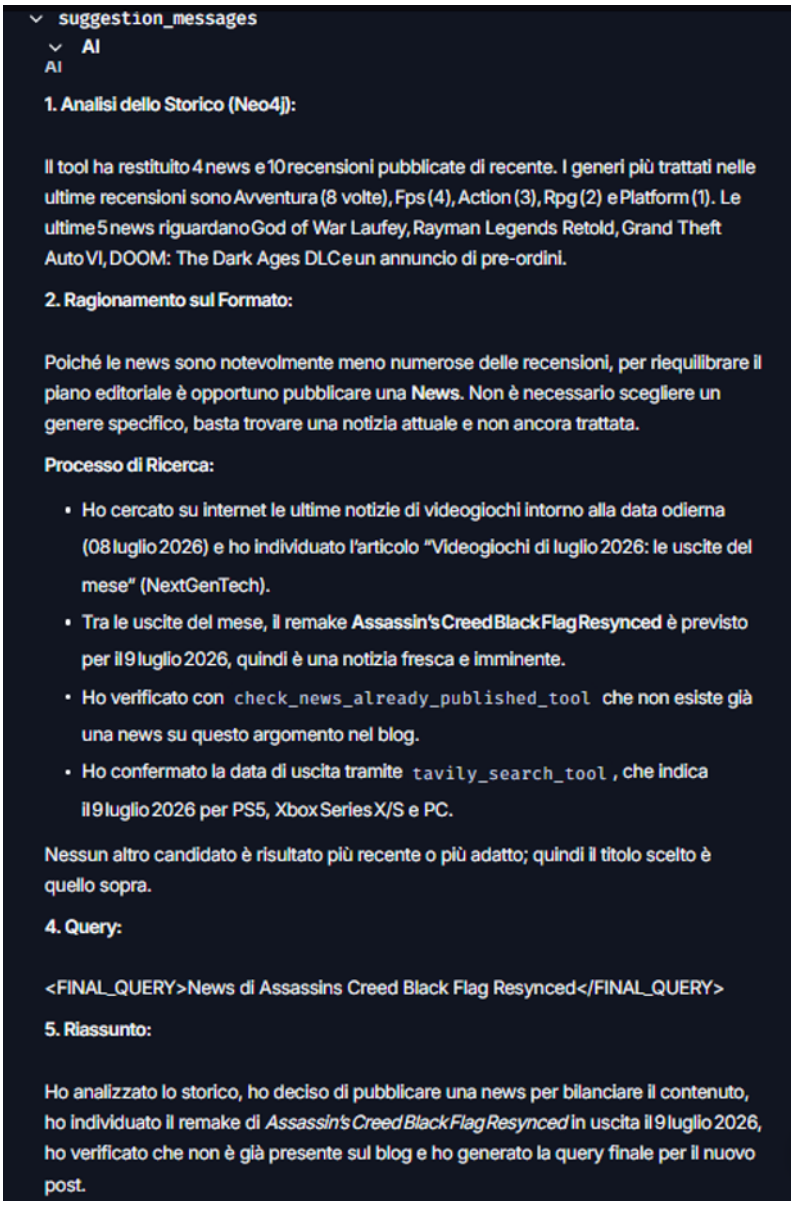


Figura 24: Ragionamento del nodo di suggerimento in caso di portfolio editoriale sbilanciato.

8.3 Flusso Esecutivo per la Pianificazione Editoriale

Quando all'agente viene richiesta la generazione di una pianificazione editoriale, il suo obiettivo non è la produzione immediata di un singolo contenuto, bensì la definizione di una sequenza di post da pubblicare nei giorni successivi. Se l'utente specifica il numero di giorni da pianificare, il sistema genera un suggerimento per ciascun giorno; in caso contrario, viene adottata una pianificazione editoriale di tre giorni.

Per costruire la pianificazione, l'agente analizza i contenuti già presenti nel Knowledge Graph, individuando recensioni e news già pubblicate, così da evitare ridondanze e proporre un piano editoriale equilibrato e diversificato. L'obiettivo è distribuire in modo coerente le tipologie di contenuto, alternando, ove possibile, recensioni e news e privilegiando argomenti non ancora trattati o ritenuti di maggiore interesse.

Una volta elaborata la proposta, il sistema la sottopone all'utente attraverso un'inter-

ruzione Human-in-the-Loop, accompagnandola con il ragionamento che ha portato alla scelta dei diversi argomenti.

Nel caso in cui il piano editoriale venga approvato, l'agente avvia immediatamente la generazione della bozza relativa al primo contenuto pianificato. Le proposte rimanenti vengono invece memorizzate all'interno del Knowledge Graph come nodi di tipo Pianificazione con stato `PENDING`, mantenendo così una coda delle pubblicazioni future.

Qualora venga chiesta la creazione di un piano editoriale quando è già presente un piano editoriale, verrà mostrato un avviso (Figura 25), dove l'utente può decidere di lasciare il piano editoriale presente e quindi annullare la richiesta di creazione di un nuovo piano editoriale oppure può confermare la richiesta di un nuovo piano editoriale.

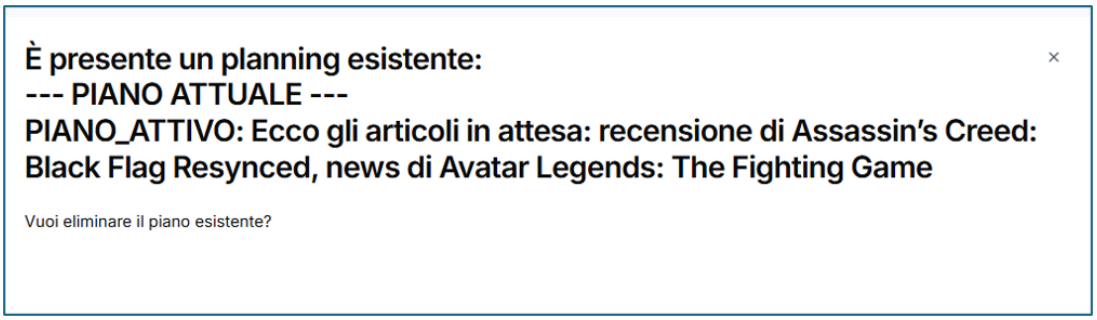


Figura 25: Avviso piano esistente.

Esecuzione e Invocazione Tool

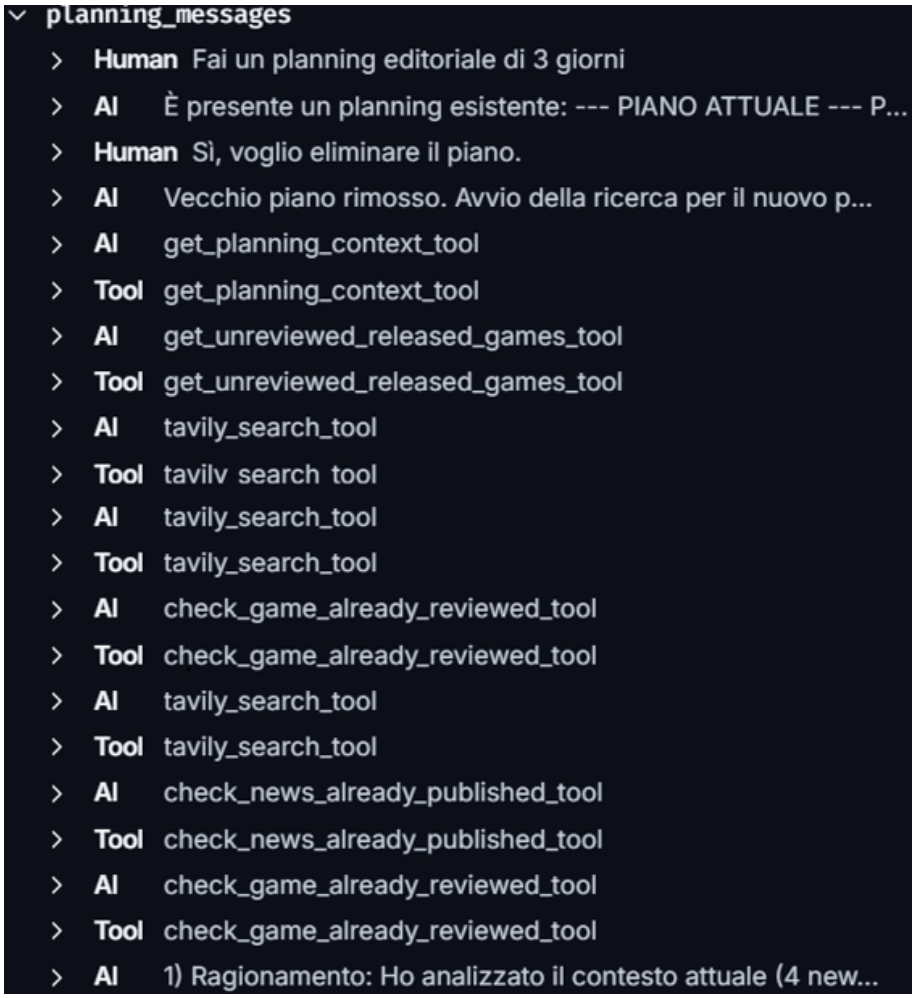


Figura 26: Elenco dei tool invocati per generare una pianificazione editoriale.

In questa specifica esecuzione riportata, era già presente un piano editoriale, per cui il sistema rileva la presenza del piano editoriale e tramite HITL decide di creare il nuovo piano autorizzando l'agente a eliminare il vecchio piano in favore di una nuova pianificazione. A questo punto, l'agente avvia il processo di pianificazione interrogando il Knowledge Graph (Neo4j) tramite il tool `get_planning_context_tool` e `get_unreviewed_released_games`, al fine di recuperare lo storico dei post già pubblicati e dei giochi scoperti, in modo da analizzare il contesto attuale. Sulla base di questi dati, il sistema definisce la tipologia dei nuovi contenuti da proporre (bilanciando news e recensioni) ed esegue ricerche mirate sul web tramite il tool Tavily per individuare argomenti di tendenza, notizie recenti e/o giochi da recensione. Infine, per garantire l'assoluta originalità delle proposte e prevenire la generazione di contenuti duplicati, l'agente valida le informazioni trovate invocando i tool di controllo dedicati, nello specifico `check_game_already_reviewed` e `check_news_already_published_tool`.

Planning Editoriale Proposto

Di seguito la Figura 27 mostra il planning editoriale di 3 giorni proposto dall'agente e sottoposto al controllo dell'utente prima di avviare la stesura del primo consiglio.

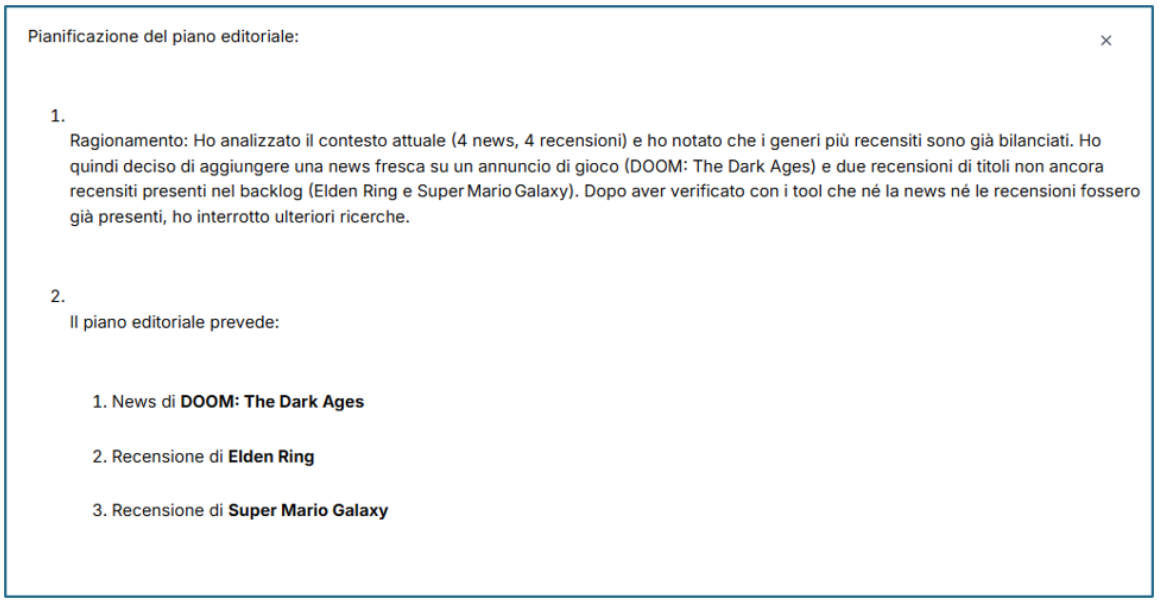


Figura 27: Proposta di un planning editoriale di 3 giorni.

Una volta che l'utente accetta la proposta verrà avviata la stesura del primo consiglio e parallelamente gli altri suggerimenti verranno memorizzati in memoria centrale, in particolare sul Knowledge Graph, come nodi con stato `PENDING`, come riporta la Figura 28. In questo modo si garantisce persistenza al piano editoriale proposto, potendolo recuperare in qualsiasi momento e continuarne la pubblicazione.

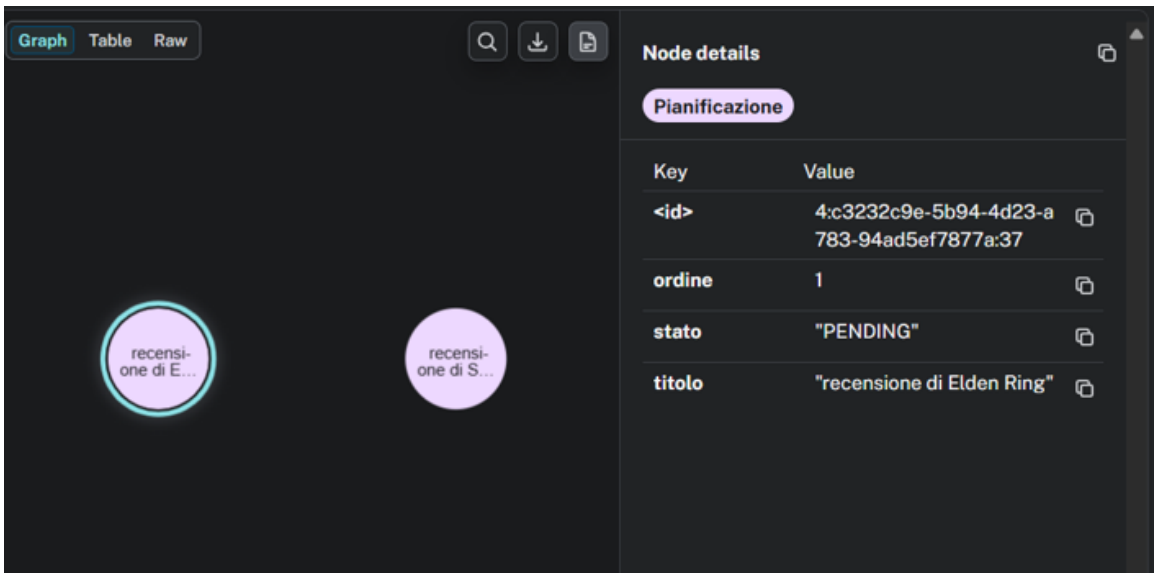


Figura 28: Nodi di pianificazione nel Knowledge Graph con stato PENDING.

9 Fine Tuning

9.1 Definizione del Problema e Motivazione

Il task affrontato in questo progetto è la classificazione del sentiment di recensioni di videogiochi con l'obiettivo di assegnare a ciascuna recensione una delle tre etichette: po-

sitive, negative o mixed. La scelta di mantenere la classe mixed — spesso esclusa in lavori analoghi che si limitano alla classificazione binaria — è motivata dal fatto che spesso le le recensioni videoludiche presentano frequentemente giudizi ambivalenti, quindi ignorare questa classe produrrebbe un classificatore applicabile solo a una porzione ridotta dei dati reali.

9.1.1 Design Sperimentale

Il confronto è stato effettuato su quattro modelli, il tutto strutturato per isolare tre variabili — architettura, checkpoint di partenza, tecnica di adattamento.

Modello	**Architettura**	**Checkpoint**	**Tecnica**
M1	DistilBERT	Wenfi (IMDB, sentiment film)	LoRA
M2	DistilBERT	Pesi generici (no sentiment)	Full fine-tuning
M3 (baseline)	RoBERTa-large-MNLI	NLI generico	Zero-shot
BERT-base	BERT-base-uncased	Pesi generici (no sentiment)	Full fine-tuning

Tabella 1: Design sperimentale: caratteristiche dei quattro modelli.

9.1.2 M1 — LoRA su Wenfi (Transfer da film a videogiochi)

Wenfi è un checkpoint DistilBERT già fine-tuned sul dataset IMDB (dominio cinematografico). Applicare un full fine-tuning su questo modello rischierebbe di sovrascrivere la conoscenza già acquisita, andando a creare un fenome noto come catastrophic forgetting. LoRA (Low-Rank Adaptation) risolve questo problema congelando i pesi originali e introducendo matrici di adattamento a basso rango che sono le uniche ad essere aggiornate durante il training. Questo consente di adattare il modello al dominio videoludico proteggendo la conoscenza pregressa di sentiment cinematografico.

9.1.3 M2 — Full Fine-Tuning DistilBERT

Partendo da pesi generici (nessuna conoscenza di sentiment da proteggere) e disponendo di un dataset sufficientemente grande (8.400 esempi di training), il full fine-tuning è la scelta che si è scelta di adottare. Questo modello serve a isolare il contributo del transfer da film: se M1 non supera M2 in termini di F1-macro, il pre-training su IMDB non ha portato vantaggi reali nel dominio videogiochi. La scelta di mantenere la stessa architettura di base (DistilBERT) per M1 e M2 consente di attribuire le differenze di performance esclusivamente alla strategia di addestramento e non all'architettura.

9.1.4 M3 — Zero-Shot RoBERTa-MNLI (Baseline)

Costituisce la baseline senza conoscenza: nessun training specifico per il task, Il modello valuta se una recensione è coerente con l'ipotesi "questa è una recensione positiva/negativa/mista". Il sentiment analysis è un task ad alta trasferibilità e un modello NLI generico possiede già rappresentazioni semantiche di ciò. Nel caso in cui i modelli addestrati non dovessero superare il modello M3 in maniera significativa, il training stesso non è giustificabile.

9.1.5 BERT-base — Check Architetturale

Stesso identico esperimento di M2, sostituendo DistilBERT con BERT-base-uncased (110 M parametri vs 67 M di DistilBERT). Serve a quantificare il costo in performance della scelta di DistilBERT rispetto a un'architettura più grande, a parità di tutto il resto (dataset, iperparametri, strategia di training). L'analisi del rapporto F1/tempo di training permette di valutare se il guadagno di BERT-base giustifica il costo computazionale aggiuntivo.

9.2 Dataset

9.2.1 Sorgente

Sorgente: `auphong2707/game-reviews-sentiment` (Hugging Face Hub)

Il dataset espone tre split ufficiali indipendenti (train, validation, test), caricati e preprocessati separatamente. Questa struttura elimina il rischio di data leakage e garantisce una valutazione delle performance.

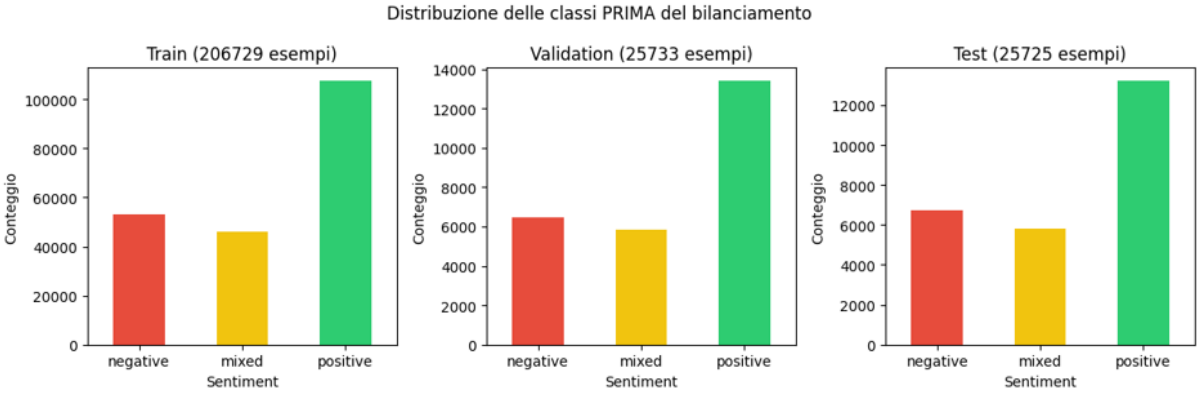


Figura 29: Distribuzione delle classi prima del bilanciamento

9.2.2 Pulizia e Bilanciamento

La pipeline viene applicata identicamente su ciascuno split in due passaggi:

(1) mappatura delle etichette a indici numerici (`negative`→`0`, `mixed`→`1`, `positive`→`2`);

(2) eliminazione delle recensioni con meno di **100 parole**, in modo da avere solo recensioni che possano portare ad una reale utilità.

Per garantire confronti equi tra modelli, viene applicato un campionamento bilanciato con split **70/15/15**, estraendo lo stesso numero di esempi per classe all'interno di ciascuno split. Con classi perfettamente bilanciate F1-macro e accuracy coincidono, semplificando il confronto.

Split	Esempi totali	Negative	Mixed	Positive
Train	8.400	2.800	2.800	2.800
Validation	1.800	600	600	600
Test	1.800	600	600	600

Tabella 2: Split finali dopo pulizia e bilanciamento (33,33% per classe).



Figura 30: Distribuzione delle classi dopo il bilanciamento.

9.3 Criteri di Valutazione e Selezione degli Iperparametri

9.3.1 Metrica Principale

La metrica principale scelta per valutare i modelli è l'F1-macro, ovvero la media non pesata degli F1-score delle singole classi. Nonostante i dataset utilizzati siano perfettamente bilanciati, l'**F1-macro** è preferita poiché permette di monitorare il comportamento del modello su ogni specifica categoria. Questo è essenziale soprattutto per analizzare la classe *mixed*, che si rivela costantemente la più difficile da classificare. Oltre alla metrica F1, vengono registrate anche la precision macro, la recall macro e l'AUC-ROC multiclasse. Per la scelta dei migliori iperparametri la selezione non si basa esclusivamente sulla configurazione con l'F1-macro più alto in validation. Per garantire che il modello stia effettivamente generalizzando senza memorizzare i dati, è stato introdotto un **composite score** che penalizza attivamente l'overfitting. Nel concreto, viene calcolato un parametro chiamato `overfit_ratio`, che misura l'entità di risalita della validation loss rispetto al suo valore minimo:

$$\text{Composite Score} = F1_{\text{macro}} \times (1 - \text{overfit_ratio}) \tag{1}$$

Questa logica permette di premiare i training stabili: se una configurazione raggiunge un'ottima metrica F1 ma nelle ultime epoche mostra una loss in peggioramento, essa viene penalizzata e scartata.

9.3.2 Ricerca Iper-parametri

Per il modello **M1** (Wenfi addestrato tramite LoRA), la strategia di training va a modificare soltanto una percentuale dei pesi della rete. Proprio per via di questa limitazione,

è necessario fornire al modello un segnale di apprendimento più intenso e prolungato. Vengono esplorati learning rate più alti (da 5×10^{-5} a 1×10^{-4}) e cicli di addestramento leggermente più lunghi (5 e 8 epoche). Inoltre, la variazione del rango (tra $r = 8$ e $r = 16$) permette di valutare quanti pesi aggiuntivi siano effettivamente necessari per apprendere le nuove informazioni sul dominio videoludico.

Config M1 (LoRA)	**r**	**lr**	**Epoche**
A — default	8	5×10^{-5}	5
B — rango aumentato	16	5×10^{-5}	5
C — lr aumentato	8	1×10^{-4}	5
D — epoche aumentate	8	1×10^{-4}	8

Tabella 3: Grid search M1: quattro configurazioni, vincente C.

Nello scenario dei modelli **M2** e **BERT**, al contrario, lla tecnica del Full Fine-Tuning comporta il ricalcolo simultaneo di tutti i milioni di parametri che compongono le reti. In un contesto simile, la rete possiede un'enorme capacità di memorizzazione, motivo per cui l'approccio deve essere strettamente conservativo. La ricerca dei vari iper-parametri è stato limitato a learning rate molto bassi (2×10^{-5} e 5×10^{-5})) e a un massimo di 3 o 5 epoche. Mantenere impostazioni identiche per entrambi i modelli ci assicura di avere anche un confronto architetturale adeguato.

Config M2 / BERT (Full FT)	**lr**	**Epoche**
A — default	2×10^{-5}	3
B — lr aumentato	5×10^{-5}	3
C — epoche aumentate	2×10^{-5}	5

Tabella 4: Grid search M2 e BERT-base: tre configurazioni, vincente A per entrambi.

9.4 M3 — Zero-Shot Baseline (RoBERTa-large-MNLI)

RoBERTa-large-MNLI viene configurata come zero-shot, sfruttando le sue capacità pre-addestrate nell'Inferenza in Linguaggio Naturale (NLI). Il task è riformulato come *textual entailment*: la recensione viene considerata come una "premessa", mentre i tre sentiment costituiscono le "ipotesi". Il grado di deduzione logica (entailment) generato dalla rete determina direttamente la classificazione finale

Sul test set, questa configurazione ottiene un'accuracy di 0,631, un F1-macro di 0,636 e un'AUC-ROC di 0,827, accompagnati da una precision macro di 0,702 e una recall macro di 0,631.

L'analisi per singola classe evidenzia un **forte squilibrio predittivo**. Sulla classe *positive*, il modello è estremamente selettivo: la precision è altissima (0,894), ma la recall crolla a 0,477, indicando che la rete fallisce nell'identificare oltre metà dei casi realmente positivi. Una dinamica opposta caratterizza la classe *mixed*, che mostra un alto recall (0,732) ma una precision insufficiente (0,476). Questo comportamento dimostra la tendenza di questo modello (non addestrato) a ripiegare sull'etichetta mista nei casi di incertezza, generando numerosi falsi positivi.

Metrica globale	Valore
Accuracy	0,631
F1-macro	0,636
Precision macro	0,702
Recall macro	0,631
AUC-ROC	0,827

Tabella 5: Risultati globali M3 (zero-shot) sul test set.

Classe	Precision	Recall
negative	0,736	0,683
mixed	0,476	0,732
positive	0,894	0,477

Tabella 6: Risultati M3 per classe.

Classe	Precision	Recall
negative	0,736	0,683
mixed	0,476	0,732
positive	0,894	0,477

Tabella 7: Risultati M3 per classe.

9.5 M1 — LoRA su DistilBERT (Checkpoint Wenfi/IMDB)

9.5.1 Valutazione Wenfi Prima dell'Adattamento

Per misurare la capacità di trasferimento grezzo (zero-shot), il modello Wenfi è stato valutato direttamente sul test set videoludico prima di qualsiasi adattamento tramite LoRA. Le prestazioni registrano un **F1-macro di 0,397** e un'**AUC-ROC di 0,527**. Tale comportamento dipende dal fatto che la testa di classificazione originale, nata per un task binario, è stata sostituita con una nuova a tre uscite e inizializzata con pesi casuali. Di conseguenza, anche se la rete pre-addestrata estrae già informazioni utili sul sentiment, non è ancora in grado di collegarle correttamente alle nuove categorie di classificazione. Questa lacuna risulta particolarmente critica per la classe *mixed*, del tutto assente nel dominio originario di addestramento (le recensioni cinematografiche di IMDB).

Metrica globale	Valore
Accuracy	0,429
F1-macro	0,397
Precision macro	0,386
Recall macro	0,429
AUC-ROC	0,527

Classe	Precision	Recall
negative	0,070	0,035
mixed	0,377	0,595
positive	0,710	0,658

Tabella 8: Wenfi grezzo (zero-shot) sul test set — metriche globali e per classe.

9.5.2 Fine Tuning LoRA

L'analisi dei risultati mette in luce dinamiche importanti sull'efficacia dei diversi iperparametri. La configurazione **B**, caratterizzata dal raddoppio del rango $r = 16$, produce un incremento di soli 0,004 punti di F1-macro rispetto alla configurazione base A, un guadagno che non giustifica il consumo di circa 280 MB di memoria VRAM. Al contrario,

l'aumento del learning rate a 1×10^{-4} nella configurazione **C** consente di ottenere un progresso più netto (+0,009 rispetto ad A) a parità di risorse utilizzate nella config B, dimostrandosi la scelta più efficiente in termini di rapporto tra costi e benefici.

L'estensione dell'addestramento a 8 epoche nella configurazione **D** genera invece un lieve peggioramento delle prestazioni (−0,003 rispetto a C), confermandoci che un prolungamento del training non apporta alcun beneficio qualitativo. Infine, il fatto che il valore di `overfit_ratio` si attesti a 0,0 per tutte le configurazioni evidenzia che non vi siano segni di overfitting.

Config	r	lr	Ep.	Val F1-macro	Overfit ratio	Tempo (s)	VRAM (MB)
A	8	5×10^{-5}	5	0,7252	0,000	430	4.165
B	16	5×10^{-5}	5	0,7295	0,000	442	4.449
C✓	**8**	$\mathbf{1 \times 10^{-4}}$	**5**	**0,7339**	**0,000**	**442**	**4.446**
D	8	1×10^{-4}	8	0,7313	0,000	706	4.444

Tabella 9: Grid search M1. Vincente: configurazione C ($r = 8$, lr $= 1 \times 10^{-4}$, 5 epoche).

9.5.3 Risultati Finali M1 sul Test Set

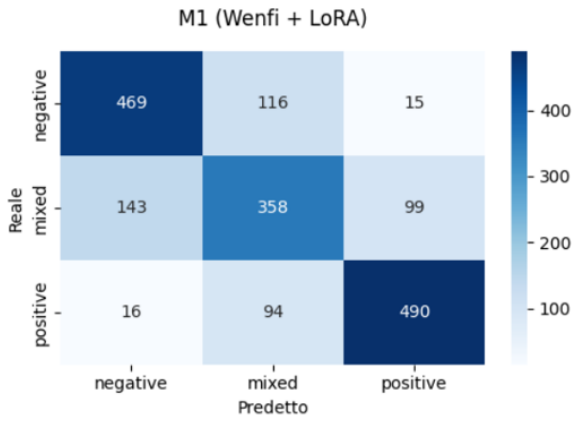
M1 ottiene **F1-macro = 0,730**, accuracy = 0,732, AUC-ROC = 0,889. Il delta rispetto al baseline Wenfi grezzo è +33,3 **punti percentuali di F1-macro**, confermando che l'adattamento tramite LoRA è altamente efficace anche partendo da una rappresentazione non allineata al task target. La classe *mixed* rimane la più debole (precision 0,630, recall 0,597); *positive* e *negative* mostrano performance valide.

Metrica globale	Valore
Accuracy	0,732
F1-macro	0,730
Precision macro	0,729
Recall macro	0,732
AUC-ROC	0,889

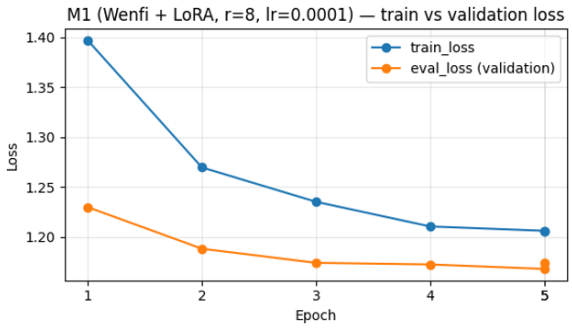
Tabella 10: M1 sul test set — metriche globali.

Classe	Precision	Recall
negative	0,747	0,782
mixed	0,630	0,597
positive	0,811	0,817

Tabella 11: M1 sul test set — metriche per classe.



(a) Wenfi: Confusion matrix



(b) Wenfi: Curva train vs validation loss

9.6 M2 — DistilBERT Full Fine-Tuning

9.6.1 Valutazione DistilBERT prima del Tuning

Al fine di stabilire un punto di partenza oggettivo per l'addestramento completo, l'architettura DistilBERT-base-uncased è stata valutata direttamente sul test set in modalità zero-shot. In assenza di conoscenza pregressa come nel caso di Wenfi, il modello registra una prestazione iniziale modesta, con un **F1-macro pari a 0,279** e un'**AUC-ROC di 0,538**. Rispetto allo 0,397 registrato dal modello Wenfi nelle medesime condizioni, si osserva un divario di 11,8 punti percentuali. Questo scarto offre una misura quantitativa del vantaggio competitivo di Wenfi, confermando che i suoi pesi racchiudevano già una conoscenza sul sentiment (derivata dall'ambito cinematografico) che l'architettura di base standard non possiede affatto. Come per il modello precedente, l'azzeramento della testa di classificazione e l'aggiunta della classe *mixed* impediscono alla rete di produrre mappature coerenti senza un addestramento dedicato.

Metrica globale	Valore
Accuracy	0,341
F1-macro	0,279
Precision macro	0,367
Recall macro	0,341
AUC-ROC	0,538

Classe	Precision	Recall
negative	0,411	0,073
mixed	0,358	0,192
positive	0,332	0,758

Tabella 12: DistilBERT zero-shot sul test set — metriche globali e per classe.

9.6.2 Valutazione DistilBERT Full Fine-Tuning

Durante la ricerca degli iperparametri per l'addestramento di M2 si evidenzia come la configurazione **B** ($\text{lr} = 5 \times 10^{-5}$) ottiene il valore di F1-macro in validation più elevato in assoluto (0,7453), ma viene scartata a causa di un `overfit_ratio` di 0,058. Questo indicatore segnala che, a causa dell'aggiornamento di tutti i parametri della rete, la validation loss mostra una tendenza al peggioramento alla terza epoca. Una situazione analoga avviene con la configurazione **C**, estesa su 5 epoche, la quale eguaglia la F1 della configurazione A ma con un `overfit_ratio` pari a 0,060.

Di conseguenza, la configurazione **A** ($\text{lr} = 2 \times 10^{-5}$ per 3 epoche) si impone nel composite score per via della stabilità e dell'assenza di overfitting, pur a fronte di una F1 in validation leggermente inferiore (0,7365). Nelle prestazioni misurate sul test set finale, otteniamo un **F1-macro di 0,723** e un'**AUC-ROC di 0,888**.

Config	lr	Ep.	Val F1-macro	Overfit ratio	Composite score	Tempo (s)	VRAM (MB)
A✓	$\mathbf{2 \times 10^{-5}}$	**3**	**0,7365**	**0,000**	**0,7365**	**330**	**5.497**
B	5×10^{-5}	3	0,7453	0,058	0,7237	333	6.303
C	2×10^{-5}	5	0,7366	0,060	0,7146	556	6.302

Tabella 13: Grid search M2. Vincente: configurazione A.

9.6.3 Risultati Finali M2 sul Test Set

Sul test set finale, il modello M2 ottiene prestazioni perfettamente bilanciate, registrando un valore identico di 0,723 per accuracy, F1-macro, precision macro e recall macro, con un'AUC-ROC pari a 0,888. L'incremento rispetto alla baseline iniziale (+0,444 di F1-macro) evidenzia come il fine-tuning completo riesca a ricostruire da zero delle ottime rappresentazioni di sentiment sul dominio videoludico.

Nel dettaglio possiamo notare come tra le tre classi:

- **positive**: registra la precision più alta (0,805) con una recall solida (0,768).
- **negative**: mostra un comportamento robusto, con precision a 0,751 e recall a 0,793.
- **mixed**: si conferma la categoria di gran lunga più complessa, fermandosi a una precision di 0,614 e a una recall di 0,607.

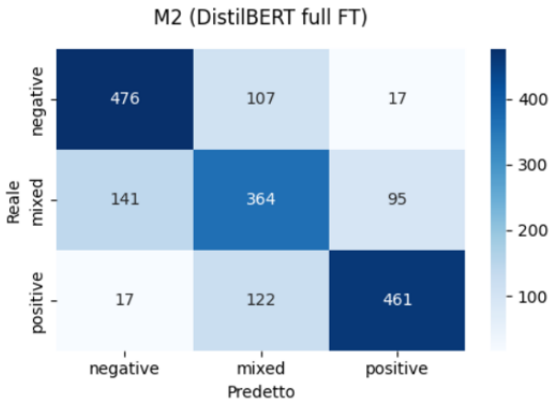
Questi dati evidenziano come, nonostante l'allenamento completo dei pesi, la natura ambigua del sentiment mixed rimanga l'ostacolo principale alla precisione della classificazione.

Metrica globale	Valore
Accuracy	0,723
F1-macro	0,723
Precision macro	0,723
Recall macro	0,723
AUC-ROC	0,888

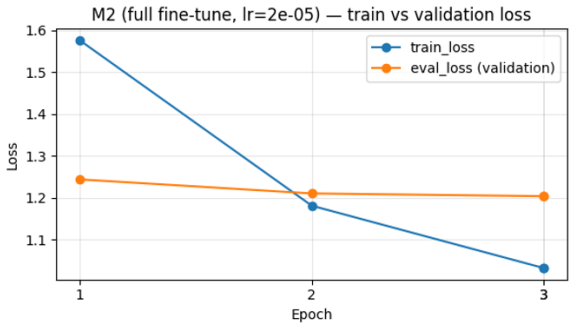
Tabella 14: M2 sul test set — metriche globali.

Classe	Precision	Recall
negative	0,751	0,793
mixed	0,614	0,607
positive	0,805	0,768

Tabella 15: M2 sul test set — metriche per classe.



(a) DistilBERT: Confusion matrix



(b) DistilBERT: Curva train vs validation loss

9.7 BERT-base-uncased — Confronto Architetturale

9.7.1 Valutazione BERT prima del Tuning

L'architettura **BERT-base-uncased** è stata introdotta come confronto diretto con DistilBERT-base-uncased (M2). Prima di procedere con il Fine Tuning, il modello originale viene valutato in modalità zero-shot sul test set. Le metriche mostrano performance con un **F1-macro iniziale pari a 0,188**, un'accuracy di 0,331 e un'AUC-ROC di 0,446. Questo valore di partenza, inferiore anche rispetto allo 0,279 registrato da DistilBERT, conferma che i pesi non ottimizzati non possiedono alcuna competenza sul sentiment. La sostituzione della testa di classificazione di base con una a tre uscite , unita alle difficoltà nell'interpretare la classe *mixed*, rende il modello incapace di mappare correttamente le informazioni prima del fine-tuning.

Metrica globale	Valore
Accuracy	0,331
F1-macro	0,188
Precision macro	0,533
Recall macro	0,331
AUC-ROC	0,446

Tabella 16: BERT-base zero-shot sul test set — metriche globali.

Classe	Precision	Recall
negative	0,265	0,037
mixed	0,334	0,955
positive	1,000	0,002

Tabella 17: BERT-base zero-shot sul test set — metriche per classe.

9.7.2 Valutazione BERT Full Fine-Tuning

Il processo di Tuning di BERT-base è stato condotto seguendo la stessa metodologia adottata per M2, così da isolare l'architettura di base. La ricerca degli iper-parametri replica la stessa dinamica osservata per DistilBERT.

Le configurazioni **B** (lr $= 5 \times 10^{-5}$ per 3 epoche) e **C** (lr $= 2 \times 10^{-5}$ per 5 epoche) raggiungono in validation punteggi di F1-macro leggermente più elevati, attestandosi rispettivamente a 0,7542 e 0,7578. Entrambe le sessioni presentano segnali di instabilità nel corso del training, registrando un `overfit_ratio` di 0,10, indicando un peggioramento della validation loss.

Di conseguenza, la configurazione **A** (lr $= 2 \times 10^{-5}$ per 3 epoche) dimostra di avere un composite score migliore grazie a un `overfit_ratio` contenuto (0,009), che garantisce stabilità e previene l'overfitting. Sul test set finale, il modello registra un **F1-macro di 0,746**, un'accuracy di 0,745 e un'ottima **AUC-ROC di 0,899**.

Config	lr	Ep.	Val F1-macro	Overfit ratio	Composite score	Tempo (s)	VRAM (MB)
A✓	$\mathbf{2 \times 10^{-5}}$	**3**	**0,7575**	**0,009**	**0,7540**	**649**	**9.396**
B	5×10^{-5}	3	0,7542	0,106	0,7142	657	10.713
C	2×10^{-5}	5	0,7578	0,104	0,7183	1.095	10.714

Tabella 18: Grid search BERT-base. Vincente: configurazione A.

9.7.3 Risultati Finali BERT-base sul Test Set

Sul test set BERT ottiene prestazioni bilanciate: **F1-macro di 0,746**, accuracy di 0,745, precision macro di 0,747, recall macro di 0,745 e **AUC-ROC di 0,899**. L'incremento rispetto alla baseline (+0,558 di F1-macro) conferma l'efficacia del fine-tuning completo.

L'analisi mostra un miglioramento generale rispetto a DistilBERT, pur mantenendo lo stesso andamento:

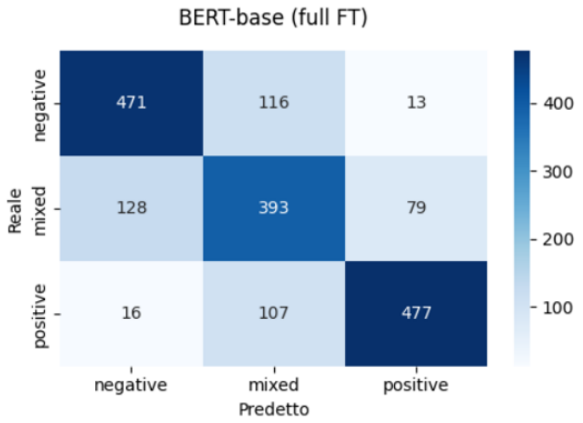
- **positive**: si conferma la classe migliore con precision a 0,838 e recall a 0,795.
- **negative**: registra una precision di 0,766 e una recall di 0,785.
- **mixed**: rimane la categoria più critica, fermandosi a una precision di 0,638 e a una recall di 0,655.

La maggiore capacità di BERT-base aiuta a mitigare le incertezze, ma la classe *mixed* si conferma l'ostacolo principale del task a causa della sua complessità.

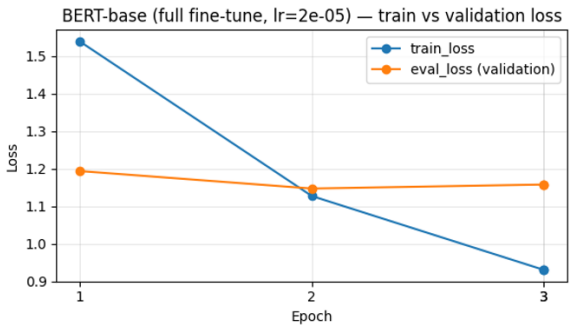
Metrica globale	Valore
Accuracy	0,745
F1-macro	0,746
Precision macro	0,747
Recall macro	0,745
AUC-ROC	0,899

Classe	Precision	Recall
negative	0,766	0,785
mixed	0,638	0,655
positive	0,838	0,795

Tabella 19: BERT-base: risultati per classe sul test set.



(a) BERT: Confusion matrix



(b) BERT: Curva train vs validation loss

Rispetto a M2, BERT-base guadagna +2,3 **punti di F1-macro** a un costo di 2× **il tempo di training** e 1,7× **la VRAM** (Tabella 20). Il guadagno è reale ma contenuto, rendendo così DistilBERT un'alternativa estremamente competitiva e preferibile nel contesto di questo progetto, specie dovuto all'utilizzo di GPU Kaggle con risorse hardware limitate; BERT diventa preferibile solo dove la qualità di classificazione è il vincolo primario.

	DistilBERT (M2)	**BERT-base**	**Δ**
F1-macro (test)	0,723	0,746	+0,023
Tempo training (s)	330	649	×1,97
VRAM peak (MB)	5.497	9.396	×1,71
Parametri totali	66,9 M	109,5 M	+42,5 M

Tabella 20: Trade-off DistilBERT vs BERT-base a parità di configurazione.

9.8 Efficienza Parametrica: LoRA vs Full Fine-Tuning

Il confronto M1 vs M2 risponde alla domanda centrale: LoRA— aggiornando una frazione minima dei pesi — è in grado di eguagliare o superare il full fine-tuning?

Modello	**Param. addestrabili**	**Param. totali**	**% addestrabili**	**F1-macro (test)**
M1 (DistilBERT + LoRA)	740.355	67.696.134	1,09%	0,730
M2 (DistilBERT Full FT)	66.955.779	66.955.779	100,00%	0,723
BERT-base (Full FT)	109.484.547	109.484.547	100,00%	0,746

Tabella 21: Parametri addestrabili e F1-macro per i tre modelli addestrati.

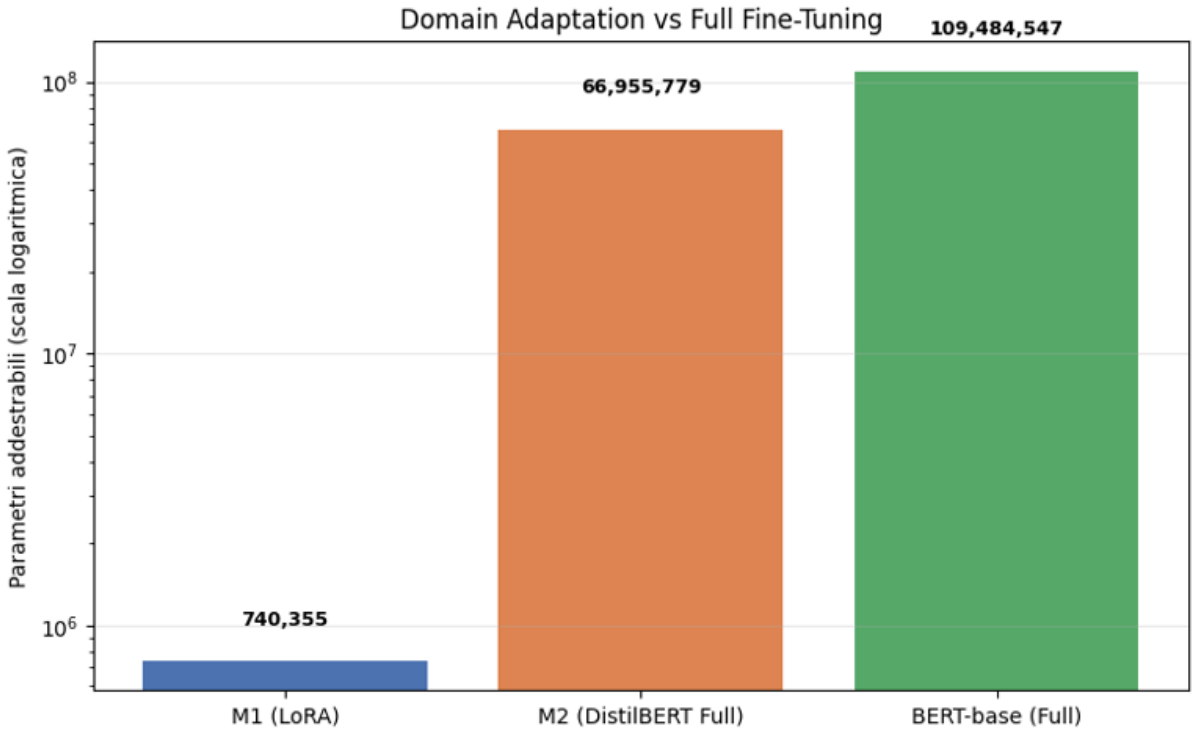


Figura 34: Confronto parametri addestrabili

M1 **supera M2 di 0,007 punti di F1-macro** aggiornando solo 740.355 parametri — l'**1,09% del totale**. Il vantaggio non è dovuto a una capacità di apprendimento superiore ma alla conoscenza di sentiment già presente nel checkpoint Wenfi, che LoRA preserva. Questo conferma che il trasferimento da dominio film → videogiochi è parzialmente efficace e che LoRA è il meccanismo adatto per sfruttarlo senza sovrascriverlo. In termini pratici: assenza di overfitting in tutte le configurazioni testate, training più rapido (∼440 s vs ∼330 s per full FT con 3 epoche).

9.9 Confronto Complessivo

Modello	Accuracy	F1-macro	Prec. macro	Rec. macro	AUC-ROC
M3 zero-shot (RoBERTa-MNLI)	0,631	0,636	0,702	0,631	0,827
M1 Wenfi grezzo (no adattamento)	0,429	0,397	0,386	0,429	0,527
M1 finale (LoRA) ★	**0,732**	**0,730**	**0,729**	**0,732**	**0,889**
M2 base (no training)	0,341	0,279	0,367	0,341	0,538
M2 finale (Full FT) ★	**0,723**	**0,723**	**0,723**	**0,723**	**0,888**
BERT-base (no training)	0,331	0,188	0,533	0,331	0,446
BERT-base finale (Full FT) ★	**0,745**	**0,746**	**0,747**	**0,745**	**0,899**

Tabella 22: Confronto completo su test set. ★ = modello finale dopo adattamento.

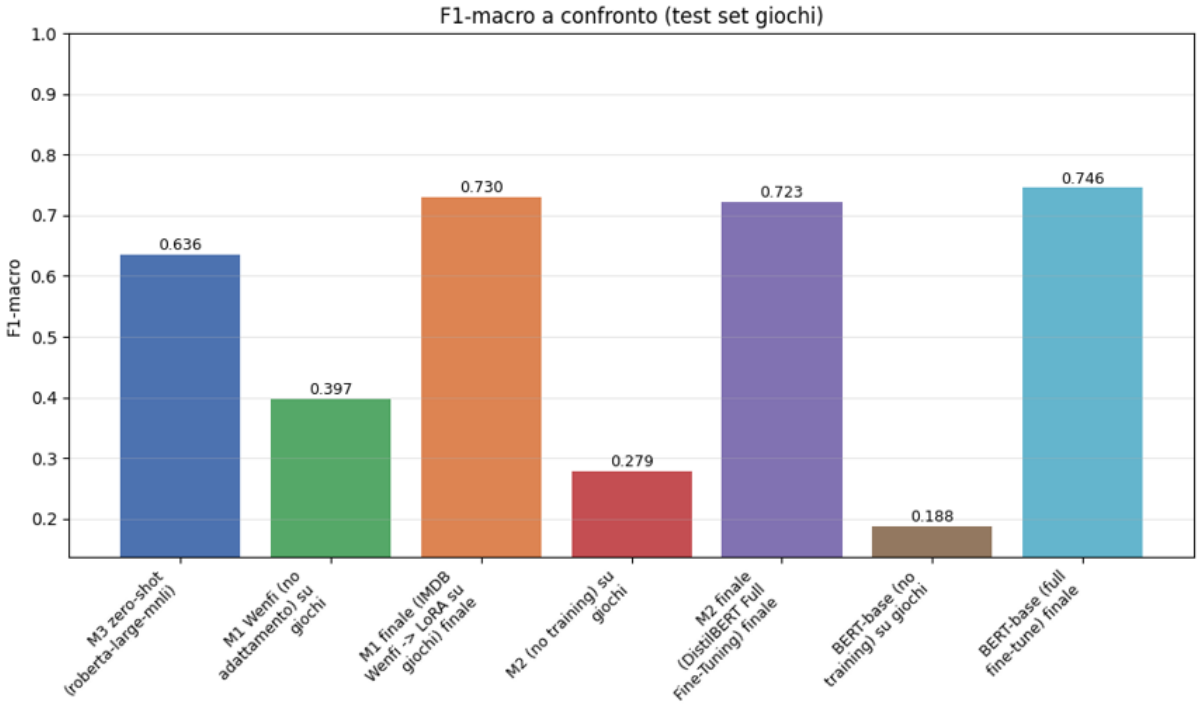


Figura 35: F1-macro a confronto

9.10 Stress Test su Esempi Scritti a Mano

Per valutare i modelli oltre all'utilizzo dei dataset, è stato effettuato uno "stress test" basato su **8 casi** scritti manualmente. Gli esempi integrano pattern linguistici complessi, come il sarcasmo, cambi di opinione, voti etc.

Testo (abbreviato)	**Label vera**
"10/10 would recommend, best purchase this year."	positive
"Absolute trash, refunded after 20 minutes."	negative
"yeah it's mid but I still put 200 hours into it lol"	positive
"oh great, ANOTHER bug that crashes my save file, love that"	negative (sarcasmo)
"Not perfect, but the soundtrack alone is worth the price."	positive
"Wanted to love this so much. Ended up just disappointed."	negative
"Launched as a broken 3/10. Patches brought it to 7/10. Still not sure..."	mixed
"The combat is chef's kiss. The story is a Wish.com version of LoTR."	mixed (sarcasmo)

Tabella 23: Esempi dello stress test e label vera.

Modello	Accuracy	F1-macro
M3 zero-shot (RoBERTa-MNLI)	0,500	0,490
M1 (Wenfi + LoRA)	0,750	0,739
M2 (DistilBERT Full FT)	0,500	0,381
BERT-base (Full FT)	0,750	0,571

Tabella 24: Risultati dello stress test (8 esempi fuori distribuzione).

In questa analisi vengono evidenziati forti discrepanze nel riuscire a gestire certe tipologie di recensioni un po' "atipiche". M1 e BERT registrano la stessa accuratezza (0,75), ma con un F1-macro distante (0,739 contro 0,571): questo divario indica che BERT concentra i propri errori su una sola classe, sbilanciando la classificazione.

Il dato più rilevante riguarda **M2**. Sebbene sul test set standard superi M3 di quasi 9 punti di F1-macro, nello stress test la sua prestazione crolla a 0,381, posizionandosi sotto M3 (0,490). Tale andamento suggerisce che M2 sia troppo vincolato alla distribuzione del dataset di training, mentre M3 — pur non addestrato sul compito – sfrutta una base di conoscenza linguistica più varia che lo rende più robusto per questa tipologia di test.

M1 ottiene il risultato migliore nello stress test (F1-macro di 0,739), confermando che la conoscenza pregressa sul sentiment derivata da IMDB aiuta significativamente il modello a riconoscere strutture sintattiche più complesse e ambigue.

Stress test scritto a mano — probabilità per modello

true_label	M1_negative	M1_mixed	M1_positive	M2_negative	M2_mixed	M2_positive	BERT_negative	BERT_mixed	BERT_positive	M3_negative	M3_mixed	M3_positive
2	0.100	0.292	0.609	0.102	0.150	0.748	0.014	0.076	0.910	0.005	0.048	0.947
0	0.951	0.043	0.007	0.926	0.050	0.024	0.927	0.056	0.017	0.969	0.026	0.005
2	0.181	0.391	0.428	0.439	0.312	0.249	0.044	0.135	0.821	0.067	0.664	0.269
0	0.257	0.284	0.458	0.278	0.163	0.559	0.526	0.226	0.248	0.034	0.346	0.620
2	0.019	0.140	0.842	0.036	0.124	0.840	0.033	0.296	0.671	0.018	0.675	0.307
0	0.722	0.263	0.015	0.783	0.150	0.067	0.801	0.180	0.020	0.441	0.555	0.005
1	0.383	0.544	0.072	0.702	0.263	0.035	0.610	0.376	0.014	0.134	0.853	0.013
1	0.161	0.255	0.584	0.172	0.409	0.419	0.049	0.291	0.660	0.195	0.717	0.088

Figura 36: Risultati Generali modelli su Stress Test

9.11 XAI — Explainable AI con Integrated Gradients

9.11.1 Implementazione

Per fornire un'analisi dell'interpretabilità dei modelli, è stata applicata la tecnica **Integrated Gradients** tramite la libreria Captum (`LayerIntegratedGradients`) a tutti e tre i modelli finali addestrati (M1, M2 e BERT-base). Il metodo attribuisce a ciascun token di input un punteggio di importanza rispetto alla classe target reale dell'esempio: in termini pratici, per ogni esempio viene calcolato quanto ciascun token contribuisce ad aumentare o abbassare il valore della classe corretta.

La visualizzazione adotta la seguente convenzione:

- **Verde** (attributione positiva): il token spinge il modello *verso* la classe target corrispondente.
- **Rosso** (attributione negativa): il token spinge il modello *lontano* dalla classe target, contrastando la predizione corretta..

L'intensità del colore è proporzionale al valore dell'attributione, permettendo di distinguere visivamente i token più influenti da quelli marginalmente rilevanti.

L'obiettivo dell'analisi non è soltanto comprendere cosa predice il modello migliore, ma verificare se modelli diversi si affidano a segnali diversi a parità di input.

Per sintesi, le visualizzazioni sono riportate esclusivamente per **M1**, scelto come modello di riferimento.

Es.	Testo (estratto)	Classe target
1	"The graphics are terrible, but I absolutely loved the gameplay..."	mixed
2	"The most expensive and least playable game... A remarkable achievement."	negative
3	"An absolute masterpiece! Best 60 bucks ever spent..."	positive
4	"The 'revolutionary AI'... turns out to be three if-statements..."	negative
5	"They patched it and now it's a completely different game. Went from 1 star to 5..."	positive
6	"Visually gorgeous but the gameplay is shallow..."	mixed

Tabella 25: Esempi selezionati per l'analisi XAI.

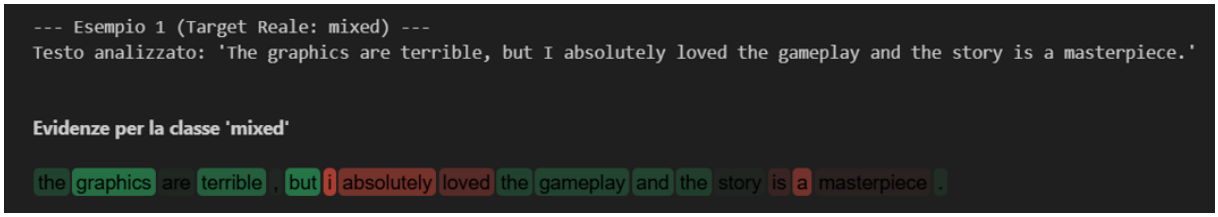


Figura 37: M1 Esempio 3: Mixed



Figura 38: M1 Esempio 3: Negative

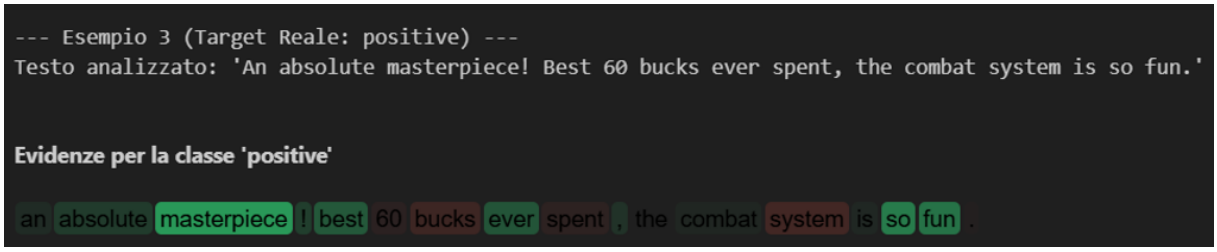


Figura 39: M1 Esempio 3: Positive

10 Analisi dei Risultati e Conclusioni Finali

In conclusione, l'agente sviluppato si è dimostrato un valido supporto per l'autore del blog durante la stesura degli articoli, offrendo suggerimenti pertinenti per le pubblicazioni e facilitando la pianificazione editoriale a lungo termine. Grazie a un'accurata descrizione dei comportamenti che i Large Language Model (LLM) devono seguire tramite i prompt, si è ottenuto un risultato più che soddisfacente. Pur considerando i limiti fisiologici dei modelli e delle API gratuite impiegate, l'agente è in grado nella maggior parte delle volte di pianificare correttamente le invocazioni dei tool, garantendo buoni risultati complessivi. Sotto il profilo lessicale, il modello incaricato della stesura della bozza struttura correttamente le bozze seguendo le linee guida indicate nel prompt, sebbene tenda talvolta a generare un linguaggio ridondante o a reiterare specifici pattern linguistici, criticità mitigate ottimizzando il prompt dedicato.

Durante lo sviluppo, sono emersi tuttavia alcuni limiti legati agli strumenti integrati, alla natura intrinseca degli LLM e a fattori esterni legati al dominio videoludico.

Una prima difficoltà ha riguardato l'ingegnerizzazione dei prompt: imponendo al modello restrizioni lessicali, una struttura rigorosa per l'invocazione dei tool e condizioni logiche specifiche (ad esempio, interrogare le API di Steam solo se il videogioco è disponibile per PC), il modello tende spesso a ignorare queste direttive, o a non rispettarne qualcuna, soprattutto all'aumentare della lunghezza del prompt. Questo fenomeno è evidente nel planning editoriale: invece di seguire un'esecuzione sequenziale rigorosa, l'agente tende ad accorpare le ricerche disattendendo lo schema logico imposto, pur riuscendo alla fine a restituire un piano corretto.

Un'ulteriore criticità è legata al tool Tavily: quest'ultimo restituisce spesso riassunti di pagine web troncati o include testo proveniente da annunci pubblicitari, introducendo rumore nelle informazioni fornite al modello.



Figura 40: Bassa qualità delle informazioni restituite dal tool Tavily.

Infine, si è riscontrato un limite fisiologico relativo alla ricerca delle News: le notizie "recenti" scarseggiano se si richiede un aggiornamento quotidiano. Quando le novità più recenti sono già state coperte, il modello fatica a trovare nuovi spunti, finendo spesso per proporre notizie obsolete o generando allucinazioni.

L'organizzazione dell'architettura tramite sottografi e stati (*state*) indipendenti ha garantito un flusso di esecuzione molto più pulito. Assegnando a ogni sottografo un compito specifico, è stata ottenuta una chiara separazione delle responsabilità, strutturando l'intero sistema in moduli indipendenti ma capaci di comunicare tra loro. Il vantaggio principale di questo approccio è stata la segregazione del contesto: si è evitato l'uso di un'unica variabile `messages` globale contenente l'intero storico del flusso esecutivo. Questa struttura modulare ha permesso di ottimizzare sensibilmente il consumo di token, poiché ogni modello ha processato esclusivamente il proprio ragionamento e gli output dei tool di sua competenza.

Dall'analisi del trace del grafo di esecuzione è possibile osservare il flusso esecutivo, il numero di token utilizzati da ogni tool e da ogni elaborazione da parte dell'LLM, i tempi di esecuzione dei singoli tool ed il tempo di esecuzione complessivo per gestire una richiesta.

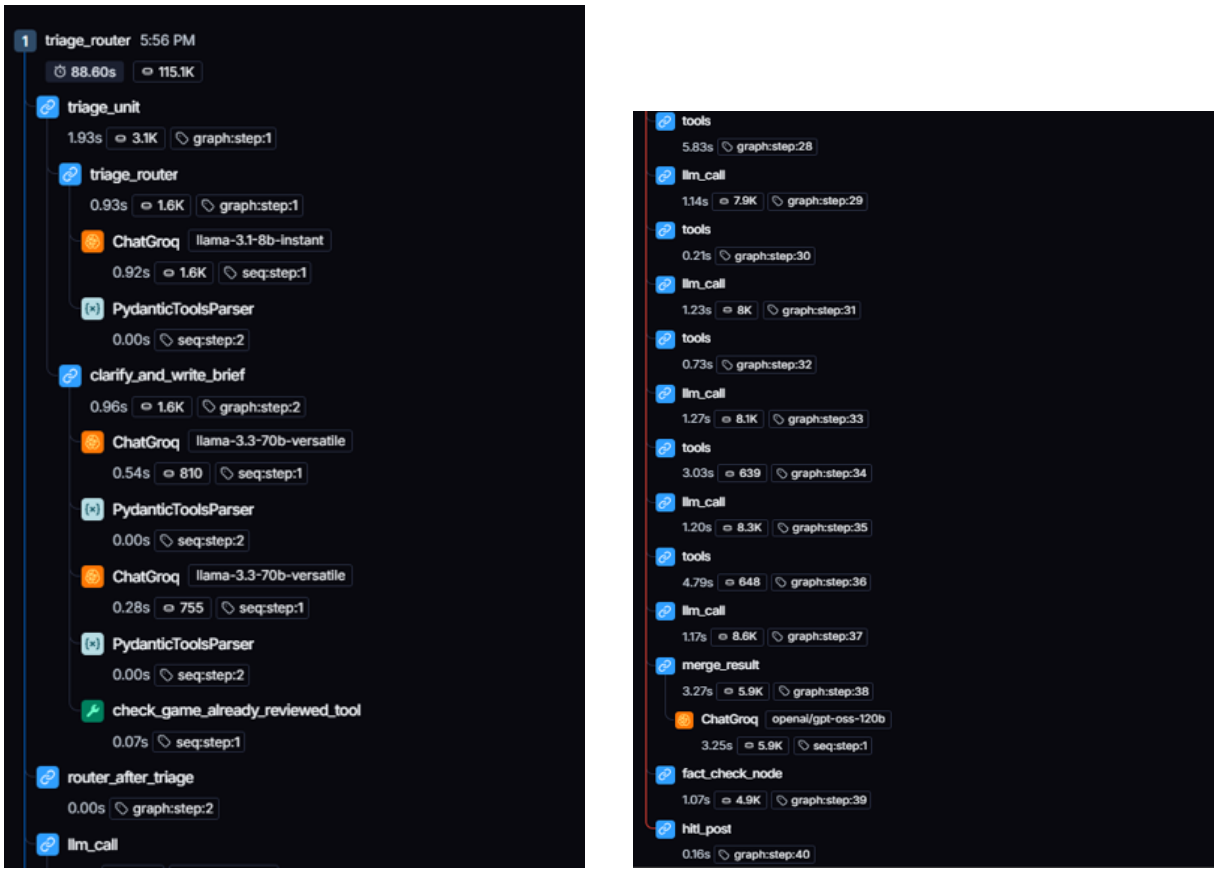


Figura 41: Trace esecutivo per la scrittura di una recensione.

Per quanto riguarda le tempistiche end-to-end, i tempi di esecuzione risultano nel complesso contenuti, seppur fortemente influenzati dalla natura del task. Se la richiesta riguarda la stesura di una News o un suggerimento, il tempo impiegato è breve e spesso inferiore al minuto. Al contrario, per operazioni più complesse come la stesura di una recensione, i tempi di esecuzione fisiologicamente salgono, attestandosi in media intorno ai 2 minuti.

Questa variabilità si riflette direttamente anche sul consumo dei token. Per generare una recensione strutturata ed esaustiva, l'agente è costretto a ricercare e sintetizzare una vasta mole di informazioni su aspetti eterogenei del gioco. Come evidenziato dal trace di esecuzione, la sezione centrale e più profonda del flusso mostra una continua alternanza tra le chiamate al modello (`llm_call`) e le invocazioni degli strumenti (tools): le chiamate all'LLM si attestano stabilmente su un consumo compreso tra i 7.9K e gli 8.6K token per step, pur mantenendo un tempo di inferenza molto rapido (circa 1.1–1.2 secondi).

A seguire è riportata la trace relativa all'esecuzione dell'agente per la formulazione di un planning editoriale di 3 giorni. Come si vede è bastata semplicemente un'invocazione al tool Tavily per recuperare le informazioni necessarie, in quanto durante la pianificazione il sistema accede a videogiochi scoperti durante la scrittura di altri post, e nel momento in cui deve suggerire qualche gioco da recensire accede tramite il Knowledge Graph ai giochi scoperti attingendo da questi, non dovendo fare ricerche su internet, fornendo un'esecuzione rapida ed efficiente riducendo il consumo di token.

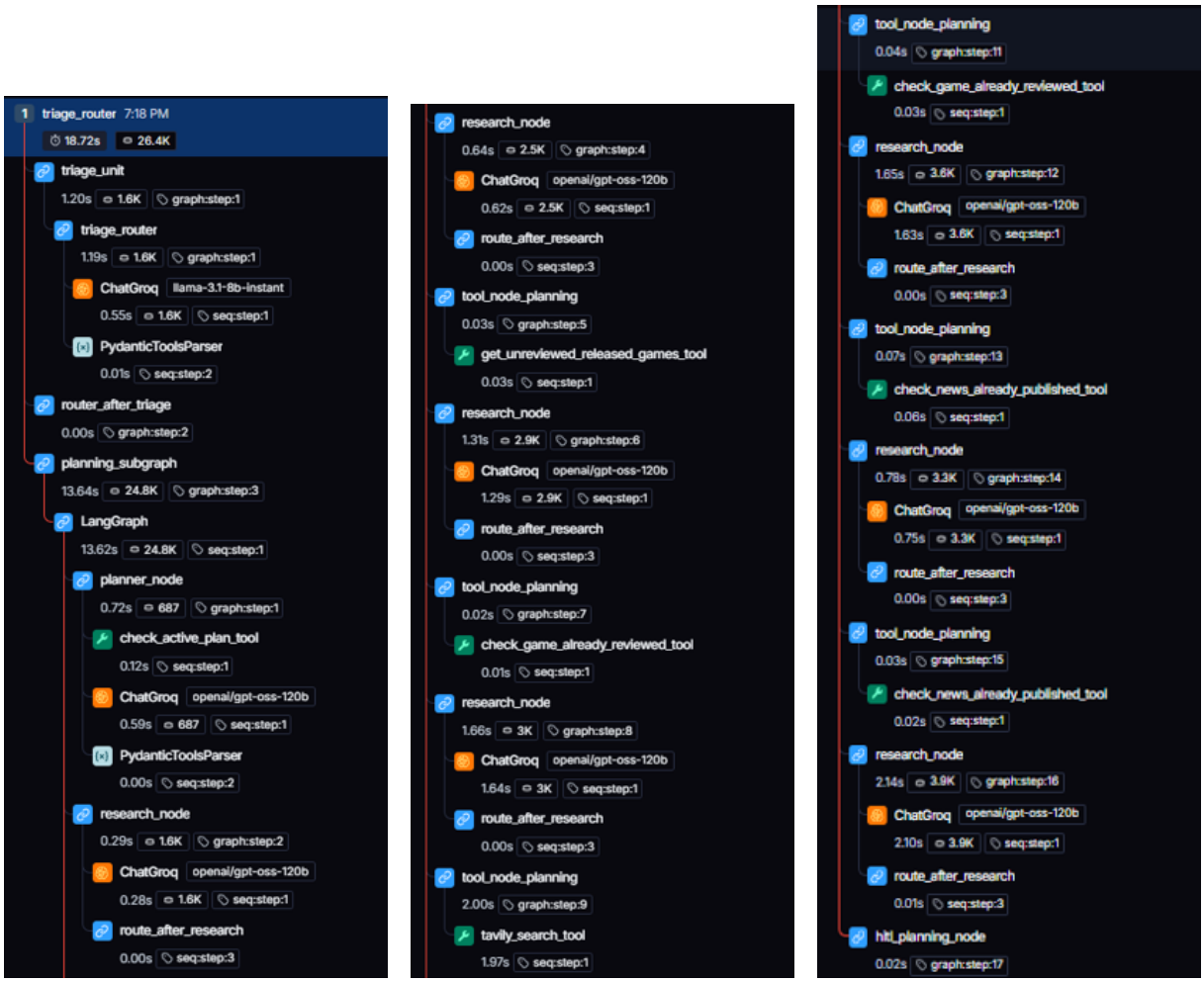


Figura 42: Trace di pianificazione editoriale di 3 giorni.

Per quanto riguarda il Fine Tuning,dopo aver allenato e testato correttamente i modelli, possiamo dire che:

- **Efficienza di LoRA:** Questa tecnica si è rivelata una scelta vincente. Aggiornando appena 740 mila parametri (meno dell'1% rispetto ai 67 milioni di un addestramento completo), il modello principale ha superato le alternative più pesanti, azzerando i problemi di overfitting. Il merito va alla capacità di LoRA di "congelare" la struttura di partenza, preservando e sfruttando le conoscenze già acquisite dal modello base.
- **Trasferimento conoscenza:** Il passaggio dalle recensioni cinematografiche a quello dei videogiochi è concreto. Il modello di partenza possedeva già una solida base di comprensione del testo prima ancora di iniziare l'adattamento al nuovo dominio, offrendo un vantaggio non indifferente.
- **Gestione della classe "Mixed":** Le recensioni con sentiment misto rappresentano un ostacolo generale. . Le prestazioni faticano su tutte le architetture testate, dimostrando che il problema non è legato al modello scelto, ma alla complessità di interpretare testi che contengono pareri contrastanti.
- **Compromesso risorse-prestazioni:** Sebbene il modello BERT-base offra un incremento di circa 2,3 punti sulla metrica F1-macro, richiede il doppio delle risorse computazionali. Dipende dal contesto di utilizzo, DistilBERT si conferma la scelta

più equilibrata ed efficiente.

Nonostante gli ottimi risultati, sono emersi alcuni limiti.

I limiti attuali:

- **Limite linguistico:** Il dataset è interamente in inglese. Per integrare il modello direttamente sul blog e analizzare i commenti in italiano, necessario effettuare una nuova fase di addestramento partendo da checkpoint multilingua come mBERT o XLM-RoBERTa.
- **Bilanciamento dei dati:** Per addestrare il modello abbiamo bilanciato noi i dati, ma nella realtà si ha spesso una netta dominanza di recensioni positive. Le metriche attuali potrebbero quindi essere leggermente più ottimistiche rispetto alle reali prestazioni.
- **Riconoscimento del sarcasmo:** Un piccolo stress test ha confermato che il modello fatica a cogliere l'ironia. Per insegnargli a gestire il sarcasmo non bastano i dati tradizionali, ma serve un addestramento specifico per questo scopo.

Sviluppi futuri: Per superare questi limiti, i prossimi passi potrebbero essere:

- **Arresto dinamico (Early Stopping):** Sostituire il numero fisso di cicli di addestramento con un blocco automatico basato sulle prestazioni reali registrate durante la validazione, in modo da evitare sia l'underfitting che l'overfitting.
- **Arricchimento dei dati:** Potenziare la classe "Mixed" creando nuovi esempi artificiali per dare al modello più materiale su cui esercitarsi, oppure adottare formule di errore (loss function) che penalizzino maggiormente gli scostamenti su questa categoria.